Министерство образования и науки России
Санкт-Петербургский Государственный Технический Университет

УДК N .....
Инв. N ...

Рукопись одобрена кафедрой .....

Зав. Кафедрой Черноруцкий И.Г.

РУКОПИСНЫЙ ФОНД
кафедры информационных и управляющих систем

Учебное пособие по курсу: "Системы программирования"

РАЗРАБОТКА ЭЛЕМЕНТОВ УЧЕБНОЙ СИСТЕМЫ ПРОГРАММИРОВАНИЯ. АБСОЛЮТНЫЙ ЗАГРУЗЧИК

Для студентов кафедры информационных и управляющих систем
Семестр 10

Автор ..... Расторгуев В.Я.

С.-Петербург

2011 г.

Министерство образования и науки России
Санкт-Петербургский Государственный Технический Университет
Кафедра информационных и управляющих систем

Расторгуев В.Я.

РАЗРАБОТКА ЭЛЕМЕНТОВ УЧЕБНОЙ СИСТЕМЫ ПРОГРАММИРОВАНИЯ. АБСОЛЮТНЫЙ ЗАГРУЗЧИК

Учебное пособие

С.-Петербург

2011 г.

АННОТАЦИЯ

Данное учебное пособие является методическим руководством при подготовке и проведению третьей из трех лабораторных работ, направленных на разработку учебной системы программирования в составе:

- компилятора с языка высокого уровня,
- компилятора с АССЕМБЛЕРА,
- абсолютного или непосредственно-связывающего загрузчика, конструктивно объединенного с эмулятором и пошаговым отладчиком объектного (target) микропроцессора, работающего на платформе, состоящей из технологического (host) микропроцессора IBM PC и одной из операционных сред MS Windows или Linux.

Целью проведения этого цикла работ является более глубокое изучение лекционного материала курса и приобретение практических навыков в процессе разработки учебной системы программирования путем адаптации программ-макетов ее элементов. Каждая из работ цикла рассчитана при соответствующей домашней подготовке в среднем на восемь часов работы в лаборатории кафедры.

СОДЕРЖАНИЕ

1. Введение 5

2. Загрузчики. Тезисы лекций по курсу 7

2.1. Схемы загрузки 7

2.1.1. Схема компиляция-выполнение 7

2.1.2. Схема абсолютной загрузки 8

2.1.3. Непосредственно – связывающий загрузчик (односекционный случай) 20

2.1.4. Непосредственно – связывающий загрузчик (многосекционный случай) 23

2.2 Классификация схем загрузки 33

2.3. Примеры статических загрузчиков 34

2.4. Динамические загрузчики 36

2.4.1. Оверлейный загрузчик 36

2.4.2. Динамически связывающий загрузчик 40

3. Рекомендации по порядку выполнения работы 43

4. Требования к содержанию отчета 44

5. Требования к комплектации, оформлению и предъявлению результатов курсовой работы 46

1. ВВЕДЕНИЕ

Данная лабораторная работа имеет своей целью получение практических навыков построения:

- загрузчика (утилиты, формирующей загрузочный эквивалент исходной программы),
- эмулятора (программной модели объектного или, иначе, целевого процессора, предназначенного для исполнения на нем загрузочного эквивалента исходной программы),
- отладчика загрузочного эквивалента исходной программы, запускаемого на эмуляторе,

являющихся элементами системы программирования, образующих в совокупности следующий технологический конвейер:

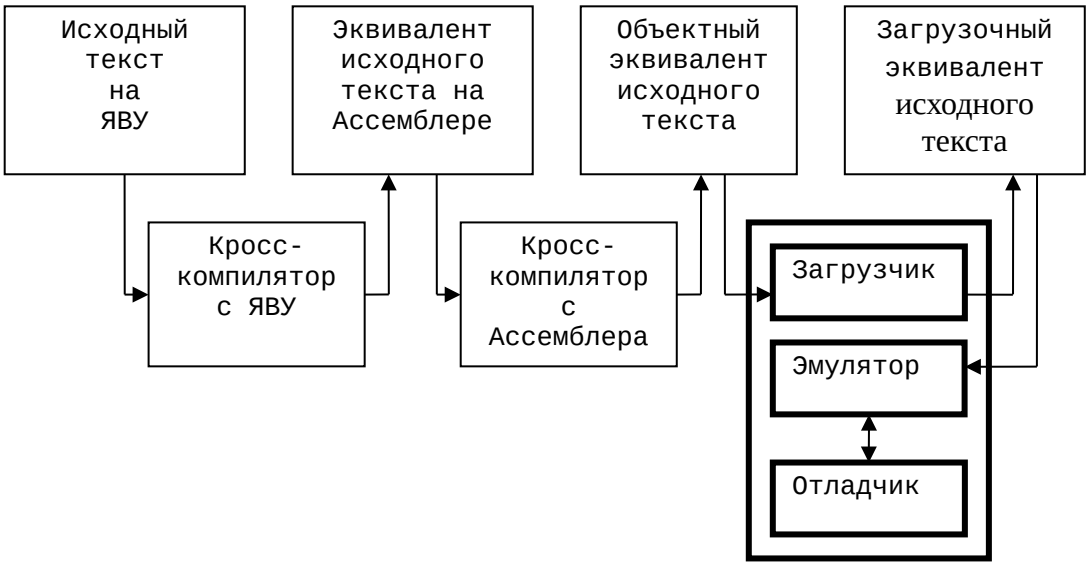

Рис.1. Состав учебной системы программирования

При этом предполагается то, что данная система программирования работает на технологической ЭВМ (IBM PC) и является по существу кросс-системой для объектной ЭВМ (IBM 370). В этой системе:

- в качестве языка высокого уровня (ЯВУ) выбран язык, образованный из подмножества языковых конструкций ПЛ1, а исходная программа готовится в виде текстового файла технологической ЭВМ с расширением *.pli,
- язык АССЕМБЛЕРА сформирован из языковых конструкций АССЕМБЛЕРА IBM 370, а ассемблеровский эквивалент исходной программы формируется в виде текстового файла технологической ЭВМ с расширением *.ass,
- объектный эквивалент исходной программы готовится в формате объектных файлов операционной системы ОС IBM 370 и хранится в виде двоичного файла технологической ЭВМ с расширением *.tex;
- загрузочный эквивалент исходной программы представляет собой машинный код IBM 370, запоминаемый в области ОЗУ

технологической ЭВМ, являющейся зоной загрузки для эмулятора объектной ЭВМ.

2. ЗАГРУЗЧИКИ. ТЕЗИСЫ ЛЕКЦИЙ ПО КУРСУ

Выходом транслятора исходных текстов, написанных на языке АССЕМБЛЕР, является объектный модуль, подготовленный в виде объектного файла, который поступает на вход загрузчика.

Загрузчик – это специальная программа системы программирования (см. Рис.1), которая подготавливает прикладную программу к выполнению и инициирует ее выполнение.

Общие задачи, требующие своего решения в процессе загрузки:

- Распределение памяти зоны загрузки под загружаемые модули.
- Разрешение ссылок на внешние символы в компануемых программах.
- Настройка переместимых (т.е. зависящих от места загрузки) величин.
- Инициализация выполнения кода, сформированного в зоне загрузки.

Если схема загрузчика позволяет решать не все из перечисленных задач, то нерешенные задачи переходят пользователю для ручного исполнения. Поэтому, рассматривая далее различные схемы загрузки, договоримся считать более совершенными те из них, которые в большей степени будут автоматизировать решение общих задач, перечисленных выше.

2.1. Схемы загрузки

2.1.1. Схема компиляция-выполнение

Особенности функционирования (см. рис.2) заключаются в том, что:

- Загрузке подлежат программы, представленные не объектными, а исходными модулями.
- Транслятор совмещает в себе как функции компилятора, так и загрузчика и выполняет действия, соответствующие этим функциям, неразрывно во времени.

Д о с т о и н с т в а:

Простота эксплуатации.

Н е д о с т а т к и:

- Сокращение пространства зоны загрузки, предназначенного для прикладной программы, из-за необходимости размещения в зоне загрузки кода самого транслятора, который имеет достаточно большой размер из-за совмещения нескольких функций.
- Необходимость избыточной трансляции исходного кода отлаженных программ в процессе их повторных загрузок и запусков для выполнения.
- Невозможность компановки в единый исполняемый код программ, написанных на различных языках программирования.

Поэтому далее стали более употребительными схемы загрузчиков (см. рис. 3), которые обладали следующими достоинствами:

- Сравнительно малый размер загрузчика оставляет больше место в зоне загрузки для исполняемого кода прикладных программ.
- Отсутствие необходимости повторной компиляции отлаженных программ.
- Возможность компановки программ, написанных на различных языках программирования.

Все схемы загрузки, которые мы рассмотрим далее, будут функционировать согласно схеме на рис. 3 и первой из них (самой простой) будет схема абсолютной загрузки.

2.1.2. Схема абсолютной загрузки

Вспомним, что объектные тексты (модули), получаемые из исходных текстов с помощью трансляторов, используются далее для формирования пакетов входных данных для загрузчика. Эти пакеты имеют структуру и состав, приведенные на рис. 4.
Схема абсолютной загрузки – основа абсолютного загрузчика предусматривает обработку только TXT-карт из всех карт входного пакета объектных модулей. Поля TXT-карт, используемые алгоритмом абсолютного загрузчика, приведены на рис. 5, а сам алгоритм – на рис. 6.

Заметим, что общие задачи загрузки (см. начало раздела) должны быть решены в процессе использования любой схемы загрузки (в том числе и абсолютной). При этом те из этих задач, которые не решаются схемой загрузки, становятся задачами пользователя схемы и должны быть решены последним вручную.

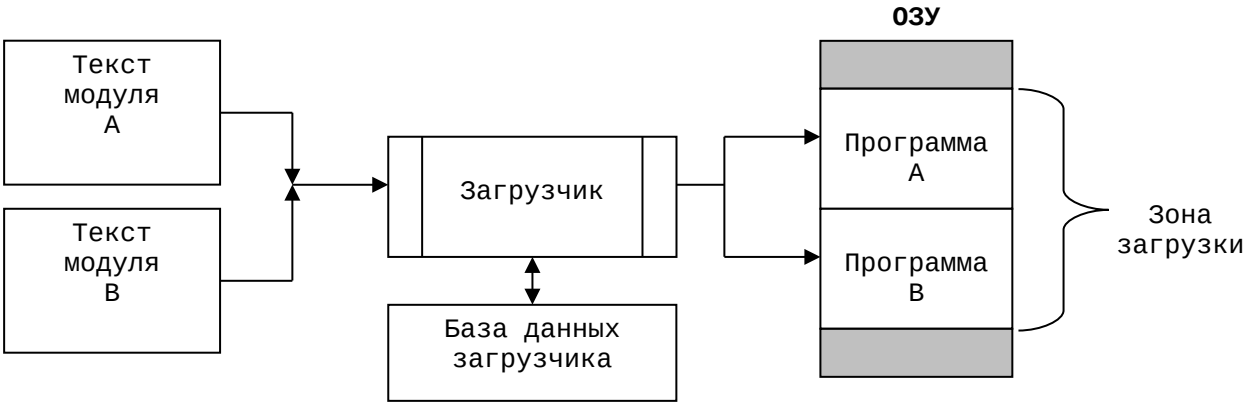

Рис.1. Общая схема функционирования загрузчика

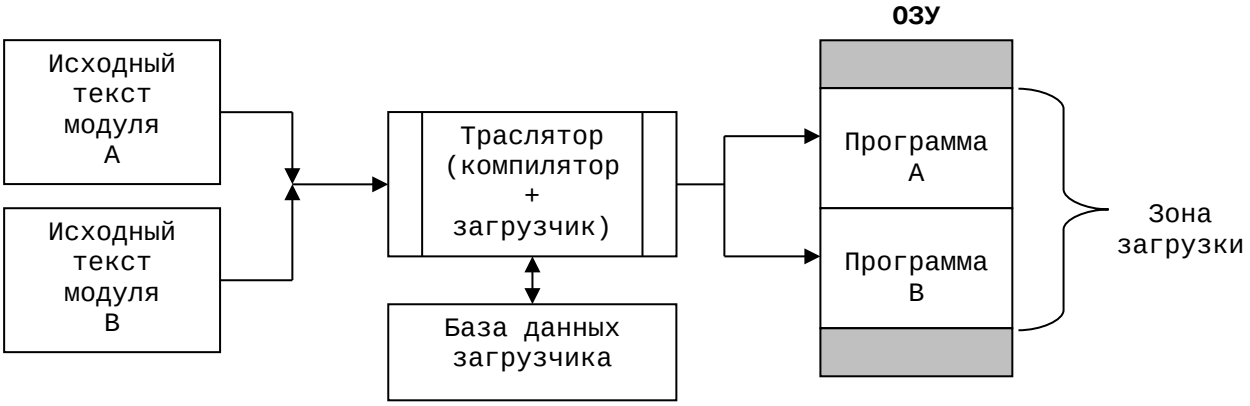

Рис.2. Функционирование схемы загрузки типа компиляция-выполнение

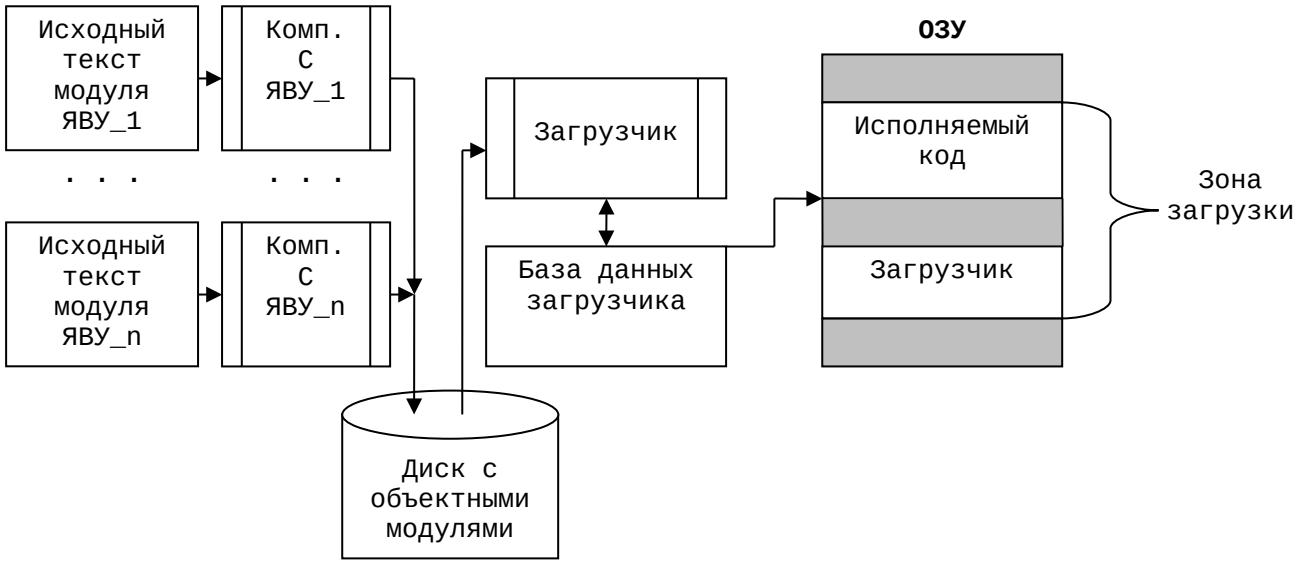

Рис.3. Схема сборки многомодульного проекта с промежуточным сохранением объектных файлов модулей

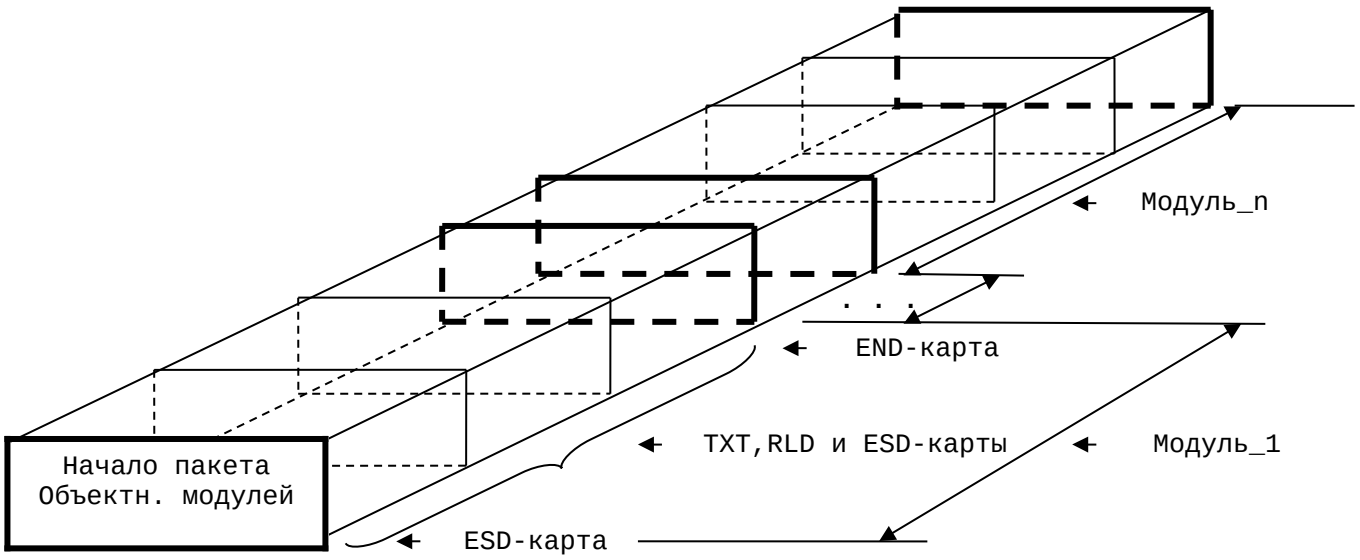

Рис.4 Состав и структура входного пакета загрузчика

Рис.5. Общий формат TXT-карты

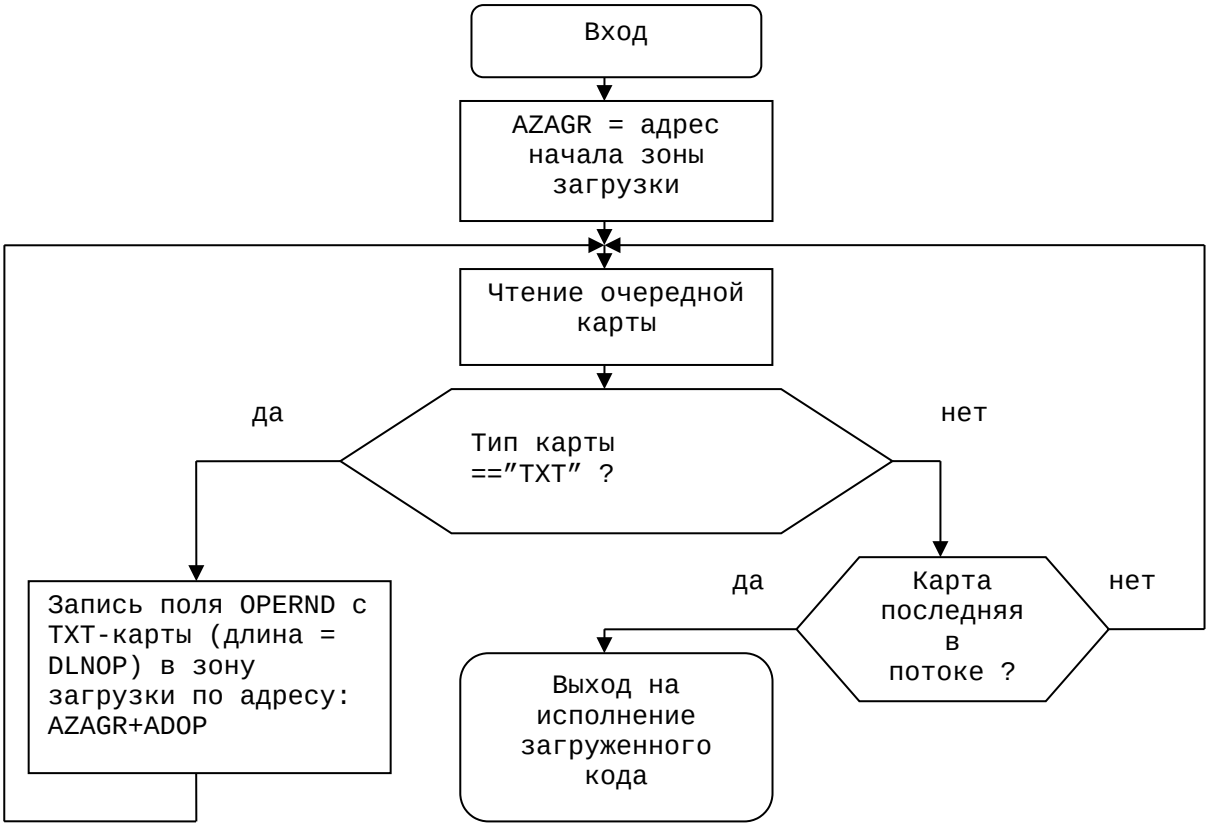

Рис.6. Блок-схема алгоритма абсолютного загрузчика

Для того, чтобы понять соотношение долей автоматического и ручного решения общих задач загрузки при использовании абсолютного загрузчика, рассмотрим пример, состоящий из вызывающей и вызываемой программы. В этом случае после двух сеансов трансляции, соответственно, программы и подпрограммы у нас в результате появятся два объектных модуля, которые могут быть сложены в общий пакет, подаваемый на вход абсолютного загрузчика.

Вызывающая программа EXAMP1

Относ. адрес	**Метка**	**КОП**	**Операнд**	**Пояснения**
0	EXAMP1	START	0	Объявить имя программы
		BALR	R15,0	R15 ← абсолютный адрес BEGCALC
2		USING	*,R15	Объявить регистр базы
	BEGCALC	ST	RVIX,RSAVE	Запомнить адрес возврата
6		L	15,AEXAMP2	РОН 15 ← абсолютный адрес нач. прогр. EXAMP2
A		BALR	RVIX,15	Передать управл. прогр. EXAMP2
C		L	RVIX,RSAVE	Восстановить адрес возврата
10		BCR	15,RVIX	Завершить программу
	R15	EQU	5	
	RVIX	EQU	14	
14	AEXAMP2	DC	F’??????????’	Абсолютный адрес начала программы EXAMP2
18	RSAVE	DS	F	Место хранения адреса возврата
1C		END		Конец исходного модуля

Использование знака вопроса “?” при определении значения поля, помеченного меткой AEXAMP2, следует понимать как признание принципиальной невозможности автоматического получения абсолютного адреса с помощью транслятора с АССЕМБЛЕРА.
Обсуждение того, как программист вручную должен рассчитать этот адрес и что он при этом должен сделать, начнем с написания вызываемой программы. Заметим при этом то обстоятельство, что, если в операторе START этой программы мы в качестве операнда установим 0, то абсолютный загрузчик (см. блок-схему алгоритма на рис.6.) запишет вызываемую программу в зону загрузки поверх ранее занесенной туда вызывающей программы. Для исключения этого программисту следует с помощью операнда оператора START сдвинуть начальный адрес вызываемой программы на величину, равную длине вызывающей программы, и выровнять полученное значение на границу ближайшего двойного слова. Это действие является по существу ручным решением, выполняемым программистом, а не абсолютным загрузчиком, который, как мы видим, не способен решать первую задачу из списка **общих задач** загрузки.

Вызываемая программа EXAMP2

Относ. адрес	**Метка**	**КОП**	**Операнд**	**Пояснения**
20	EXAMP2	START	30	Объявить имя программы
		USING	*,15	Объявить регистр базы
20		L	3,A	РОН 3 ← A
24		A	3,B	РОН 3 ← (A+B)
28		S	3,C	РОН 3 ← (A+B)-C
2C		ST	3,D	РОН 3 → D
30		BCR	15,14	Завершить программу
34	A	DC	F’3’	Переменная A
38	B	DC	F’4’	Переменная B
3C	C	DC	F’5’	Переменная C
40	D	EQU	*	Место хранения результирующей
40		DS	F	переменной D
44		END		Конец исходного модуля

В программе EXAMP1 перед обращением к программе EXAMP2 необходимо в регистр 15 загрузить абсолютный адрес начала подпрограммы EXAMP2. Как было сказано выше, при использовании абсолютного загрузчика это должен сделать вручную программист. Для этого он должен знать способ расчета абсолютного адреса начала зоны загрузки. Здесь возможен один из двух вариантов получения абсолютного адреса начала зоны загрузки в зависимости от свойств используемой операционной системы:

- Узнать абсолютный адрес начала зоны загрузки у системного администратора (в этом случае адрес имеет постоянную во времени величину).
- Узнать у системного администратора название стандартной функции (макровызова), используя которую можно получить текущее значение абсолютного адреса начала зоны загрузки.

Для простоты предположим, что мы имеем первый случай и что значение абсолютного адреса начала зоны загрузки равно, например, 20 000 (шестнадцатеричное) или 131 072 (десятичное), тогда абсолютный адрес начала вызываемой программы EXAMP2 в зоне загрузки будет равным:

Шестнадцатеричное Значение	**Десятичное значение**
20 000 + 20 = 20 020	131 072 + 32 = 131 104

Далее перенесем в программу EXAMP1 в поле AEXAMP2 полученное десятичное значение абсолютного адреса начала программы EXAMP2, внося его вместо значения “????????????” в операнд псевдооператора DC формата F.

Теперь с очевидностью можно утверждать то, что при использовании программистом абсолютного загрузчика четыре общие задачи загрузки распределяются между программистом и абсолютным загрузчиком следующим образом:

- первая (решается программистом ручным выбором значения операнда оператора START),
- вторая и третья (решаются программистом с помощью расчета, пример которого приведен выше),
- четвертая решается абсолютным загрузчиком.

Подводя итог рассмотрения абсолютного загрузчика, перечислим его достоинства и недостатки.

Д о с т о и н с т в а:

- Сравнительно малый размер загрузчика оставляет больше место в зоне загрузки для исполняемого кода прикладных программ.
- Отсутствие необходимости повторной компиляции отлаженных программ.
- Возможность компановки программ, написанных на различных языках программирования.
- Простота алгоритма.

Н е д о с т а т к и:

Низкая степень автоматизации решения большинства задач из числа общих задач загрузки и, как следствие, перекладывание на программиста значительного объема ручной работы в части обеспечения полного решения общих задач загрузки.

Вывод – абсолютный загрузчик, имея свои преимущества, не обеспечивает высокую степень автоматизации решения **общих задач загрузки**.

Перед постановкой задачи разработки новой и более совершенной в части повышения степени автоматизации решения **общих задач загрузки** схемы загрузки (назовем ее схемой **Н**е**П**осредственно-**С**вязывающего загрузчика или **НПС-**загрузчика) рассмотрим несколько соображений и примеров, которые помогут нам синтезировать перечень требований к ней. Здесь нам не обойтись без предъявления дополнительных требований, как к языку АССЕМБЛЕРА, так и транслятору с АССЕМБЛЕРА.

Дополнительные требования к языку АССЕМБЛЕРА

Для обеспечения возможности автоматического расчета абсолютного адреса, соответствующего некоторому объекту программы на АССЕМБЛЕРЕ, который помечен символом **METKA_2**:

```
          START 0
          . . .
METKA_2   объект
          . . .
          END
```

введем в язык АССЕМБЛЕРА новую модификацию псевдокоманды АССЕМБЛЕРА **DC** со спецификатором формата константы **A** вида:

```
[METKA_1] DC   A(METKA_2)
```

Появление такой псевдокоманды в программе означает то, что она должна быть заменена транслятором АССЕМБЛЕРА на абсолютный адрес, соответствующий метке METKA_2. Этот адрес должен быть сформирован в четырех байтах, левый (с меньшим адресом) из которых расположен на границе машинного слова.

Применяя новый формат псевдооператора **DC**, перепишем пример, рассмотренный выше, сначала в упрощенном (односекционном) варианте.

Относ. адрес	**Метка**	**КОП**	**Операнд**	**Пояснения**
0	EXAMP1	START	0	Объявить имя программы
		BALR	R15,0	R15 ← абсолютный адрес BEGCALC
2		USING	*,R15	Объявить регистр базы
	BEGCALC	ST	RVIX,RSAVE	Запомнить адрес возврата
6		L	15,AEXAMP2	РОН 15 ← абсолютный адрес нач. прогр. EXAMP2
A		BALR	RVIX,15	Передать управл. прогр. EXAMP2
C		L	RVIX,RSAVE	Восстановить адрес возврата
10		BCR	15,RVIX	Завершить программу
	R15	EQU	5	
	RVIX	EQU	14	
14	AEXAMP2	DC	A(EXAMP2)	Абсолютный адрес начала программы EXAMP2
18	RSAVE	DS	F	Место хранения адреса возврата
1C	EXAMP2	EQU	*	Пометить вход в подпрограмму EXAMP2
1C		L	3,A	РОН 3 ← A
20		A	3,B	РОН 3 ← (A+B)
24		S	3,C	РОН 3 ← (A+B)-C

28		ST	3,D	РОН 3 → D
2C		BCR	15,14	Завершить программу EXAMP2
30	A	DC	F’3’	Переменная A
34	B	DC	F’4’	Переменная B
38	C	DC	F’5’	Переменная C
3C	D	EQU	*	Место хранения результирующей
3C		DS	F	переменной D
40		END		Конец исходного модуля

Методика расчета абсолютного адреса

В основе методики лежит тождество, позволяющее рассчитать любой абсолютный адрес, например A(EXAMP2), по любому другому известному абсолютному адресу, например A(EXAMP1), с использованием относительного адреса. Это тождество имеет следующий вид:

$$A(EXAMP2) = A(EXAMP1) + \underbrace{[\ A(EXAMP2) - A(EXAMP1)\ ]}_{\text{Относительный адрес EXAMP2 к точке с абсолютным адресом EXAMP1}}$$

и задает способ расчета абсолютного адреса объекта, помеченного как EXAMP2, через абсолютный адрес объекта, помеченного как EXAMP1, и относительный адрес объекта с меткой EXAMP2, вычисленного относительно объекта с меткой EXAMP1.

Полный расчет абсолютного адреса должен быть завершен только к моменту начала выполнения прикладной программы. До возникновения этого момента прикладная программа должна быть последовательно обработана транслятором с АССЕМБЛЕРА и загрузчиком. Следовательно, появляется возможность разделить задачу вычисления абсолютного адреса на две части, относящиеся соответственно к транслятору и загрузчику. Транслятору логично поручить расчет относительного адреса (величины в квадратных скобках в правой части тождества), а загрузчику – окончательный расчет всей правой части тождества.

Дополнительные требования к транслятору с АССЕМБЛЕРА

Результат своего расчета относительного адреса транслятор должен оставить на TXT-карте, генерируемой в ответ на обработку строки исходного текста, содержащей определение адресной константы. В нашем примере, рассмотренном выше, эта строка исходного текста имеет вид:

DC A(EXAMP2),

а подлежащая генерации TXT-карта должна быть такой, как на рис.7:

Название полей TXT-карты → Тип карты	ADOP	DLNOP	OPERND
TXT	00 00 14	4	00 00 00 1c

Рис.7. TXT-карта

Для получения полного результата - абсолютного адреса символа EXAMP2 загрузчик должен будет далее к относительному адресу, равному 00 00 00 1C и записанному на данной TXT-карте, добавить значение абсолютного адреса символа EXAMP1. Чтобы это действие осуществилось, транслятор должен сгенерировать еще одну карту нового RLD-типа, которая имеет следующий вид:

Название полей TXT-карты → Тип карты	IDNUM	ZNAK	ADRSMESH
RLD	1	X’0C’	00 00 14

Рис.8. RLD-карта

Данная RLD-карта предписывает загрузчику добавить (ZNAK, равный X’0C’, означает сложение; для вычитания ZNAK должен быть равен X’0D’) к содержимому TXT-карты с адресом смещения 00 00 14 абсолютный адрес внешнего символа, информация о котором оставлена транслятором на ESD-карте вида:

Название полей TXT-карты → Тип карты	IDNUM	SYMBOL	SYMTYP	OADR	DLINA
RLD	1	EXAMP1	0	00 00 00	00 00 44

Рис.9. ESD-карта

Вывод: Введя в язык АССЕМБЛЕРА новую псевдокоманду вида:

[METKA_1] DC A(METKA_2)

и доработав транслятор с АССЕМБЛЕРА в части генерации RLD-карт, мы создали условия для работы НПС-загрузчика, который сможет автоматизировать решение задачи расчета абсолютных адресов помеченных локальных объектов (т.е. автоматизировать решение задачи настройки переместимых величин из списка общих задач загрузки).

Далее разберемся в том, что следует сделать для автоматизации решения задачи разрешения ссылок не только на локальные, но и на глобальные или общие символы.

Приращение дополнительных требований к языку АССЕМБЛЕР

Чтобы разобраться в том, что следует сделать для автоматизации решения задачи разрешения ссылок на общие символы, отметим, что в рассмотренном нами примере псевдокоманда

[METKA_1] DC A(METKA_2)

была использована для автоматизации расчета абсолютного адреса метки, определенной внутри программы, т.е. метки, являющейся **локальной**. Однако, часто возникает необходимость в одной программе иметь значения абсолютных адресов помеченных объектов, определенных в другой программе (т.е. иметь значения ссылок на **глобальные** метки). Вычисления абсолютных адресов, соответствующих ссылкам на **глобальные** метки, автоматизируются с помощью применения в программе, где ссылка необходима, той же самой псевдокоманды

[METKA_1] DC A(METKA_2)

но METKA_2 должна уже быть определена как глобальная ссылка с помощью дополнительной псевдокоманды **EXTRN**.

В другой же программе, содержащей объект, помеченный как METKA_2, такой объект должен быть объявлен одним из двух возможных способов:

- как глобально помеченный с помощью команды **START**,
- как локально помеченный, но превращенный в глобально помеченный с помощью специальной псевдокоманды **ENTRY**.

Воспользуемся всем, сказанным выше, на примере взаимодействия трех программ (одной вызывающей и двух вызываемых), предполагая, что вызывающая программа EXAMP1 будет определена в одноименном исходном тексте на языке АССЕМБЛЕР, а две другие – EXAMP2 и EXAMP3 будут располагаться в другом исходном тексте на языке АССЕМБЛЕРА с именем EXAMP2.

Вызывающая программа EXAMP1

Относ. адрес	Метка	КОП	Операнд	Пояснения
0	EXAMP1	START	0	Объявить имя программы
		BALR	R15,0	R15 ← абсолютный адрес BEGCALC
2		USING	*,R15	Объявить регистр базы
	BEGCALC	ST	RVIX,RSAVE	Запомнить адрес возврата
6		L	15,AEXAMP2	РОН 15 ← абсолютный адрес нач. прогр. EXAMP2
A		BALR	RVIX,15	Передать управл. прогр. EXAMP2
C		L	15,AEXAMP3	РОН 15 ← абсолютный адрес нач. прогр. EXAMP3
10		BALR	RVIX,15	Передать управл. прогр. EXAMP3
12		L	RVIX,RSAVE	Восстановить адрес возврата
16		BCR	15,RVIX	Завершить программу
	R15	EQU	5	
	RVIX	EQU	14	
18	AEXAMP2	DC	A(EXAMP2)	Абсолютный адрес начала программы EXAMP2
		EXTRN	EXAMP2	Объявление EXAMP2 внешней ссылкой
1C	AEXAMP3	DC	A(EXAMP3)	Абсолютный адрес начала программы EXAMP3
		EXTRN	EXAMP3	Объявление EXAMP3 внешней ссылкой
20	RSAVE	DS	F	Место хранения адреса возврата
24		END		Конец исходного модуля

Вызываемые программы EXAMP2 и EXAMP3

Относ. адрес	Метка	КОП	Операнд	Пояснения
0	EXAMP2	START	0	Объявить имя программы EXAMP2
		USING	*,15	Объявить регистр базы

0		L	3,A	РОН 3 ← A
4		A	3,B	РОН 3 ← (A+B)
8	CONT	S	3,C	РОН 3 ← (A+B)-C
C		ST	3,D	РОН 3 → D
10		BCR	15,14	Завершить программу
14	A	DC	F’3’	Переменная A
18	B	DC	F’4’	Переменная B
1C	C	DC	F’5’	Переменная C
20	D	EQU	*	Место хранения результирующей
20		DS	F	переменной D
24	EXAMP3	BALR	12,0	Объявить имя программы EXAMP3
		USING	*,12	Объявить второй регистр базы
26		L	15,AEXAMP2	РОН 15 ← абс. адр. EXAMP2
2A		A	3,AB	РОН 3 ← абс. адр. метки B
2E		L	3,0(0,3)	РОН 3 ← B
32		L	4,ACONT	РОН 4 → абс. адр. метки CONT
36		BCR	15,4	Передать упр. На метку CONT
38	ACONT	DC	A(CONT)	Абс. адр. Метки CONT
3C	AEXAMP2	DC	A(EXAMP2)	Абс. адр. Метки EXAMP2
40	AB	DC	A(B)	Абс. адр. метки B
44		ENTRY	EXAMP3	Объявить EXAMP3 внешним символом
44		END		

Вывод: Введя в язык АССЕМБЛЕРА новые псевдокоманды вида:

[METKA_1] ENTRY METKA_2

и

[METKA_1] EXTRN METKA_2

мы создали языковые конструкции, необходимые, соответственно, для придания локальным меткам статуса глобальных (т.е. превращать эти метки во внешние имена) и для обеспечения возможности формирования ссылок из одних программ на имена в других программах (т.е. ссылок на “чужие” внешние имена).

Приращение дополнительных требований к транслятору с АССЕМБЛЕРА

В ответ на псевдокоманды ENTRY и EXTRN транслятор должен генерировать карты ESD-типа для внешних имен соответственно типов 1 (ENTRY-случай) и 2 (EXTRN-случай). Для примера, рассмотренного выше, транслятор должен сформировать два объектных модуля, представленные на рис. 10 и рис.11. Легко видеть, что, как и прежде, записывая А-константу в виде псевдокоманды

DC A(внешнее_имя),

программист дает указание транслятору с АССЕМБЛЕРА и загрузчику рассчитать значение абсолютного адреса **внешнего_имени** и записать полученное значение в четырехбайтовое поле, зарезервированное под тело псевдокоманды.

Т.к. транслятор с АССЕМБЛЕРА при трансляции программы EXAMP1 не располагает никакой информацией о расположении внешних имен в подпрограмме EXAMP2, то его функция состоит в решении частной задачи, а именно:

- резервировании и занулении четырехбайтового поля псевдокоманды,
- формировании ESD-карты для имени EXAMP2,
- формировании RLD-карты, дающей задание загрузчику на дорасчет абсолютного адреса и занесение его в поле псевдокоманды.

Два объектных модуля вместе со списком загружаемых программ (включающим EXAMP1 и EXAMP2) являются входными данными НПС-загрузчика и входят в его базу данных.

2.1.3. Непосредственно – связывающий загрузчик (односекционный случай)

Схема функционирования НПС-загрузчика для односекционного случая, исходный текст примера которого рассмотрен выше, приведена на рис.12.

База данных НПС-загрузчика в этом случае состоит из содержимого единственного объектного модуля, т.е. из набора карт ESD, TXT, RLD и END типов, входящих в этот модуль.

Блок-схема алгоритма НПС-загрузчика представлена на рис. 13. Из нее видно, что НПС-загрузчик, соответствующий этой блок-схеме, автоматизирует решение задач с номерами 3 и 4 из числа общих задач загрузчика (см. перечень этих задач в начале раздела) для случая односекционных программ.

П р и м е ч а н и е:

Блок-схема предполагает, что в колоде карт объектного модуля TXT-карты, соответствующие псевдооперациям **DC A(метка),** опережают по порядку следования RLD-карты, предписывающие их дорасчет.

Тип	
END	

Тип	IDNUM	SYMBOL	SYMTYPE	OADR	DLINA	
ESD	3	EXAMP3	2	---	---	

Тип	IDNUM	ZNAK	ADRSMESH	
RLD	3	X’0C’	00 00 1C	

Тип	ADOP	DLNOP	OPERAND	
TXT	00 00 1C	4	00 00 00 00	

Тип	IDNUM	SYMBOL	SYMTYPE	OADR	DLINA	
ESD	2	EXAMP2	2	---	---	

Тип	IDNUM	ZNAK	ADRSMESH	
RLD	2	X’0C’	00 00 18	

Тип	ADOP	DLNOP	OPERAND	
TXT	00 00 18	4	00 00 00 00	

. . .

Тип	IDNUM	SYMBOL	SYMTYPE	OADR	DLINA	
ESD	1	EXAMP1	0	00 00 00	00 00 24	

Рис.10. Объектный модуль после трансляции исходного модуля EXAMP1

Тип	
END	

Тип	IDNUM	SYMBOL	SYMTYPE	OADR	DLINA	
ESD	2	EXAMP3	1	00 00 24	---	

Тип	IDNUM	ZNAK	ADRSMESH	
RLD	1	X’0C’	00 00 40	

Тип	ADOP	DLNOP	OPERAND	
TXT	00 00 40	4	00 00 00 18	

Тип	IDNUM	ZNAK	ADRSMESH	
RLD	1	X’0C’	00 00 3C	

Тип	ADOP	DLNOP	OPERAND	
TXT	00 00 3C	4	00 00 00 00	

Тип	IDNUM	ZNAK	ADRSMESH	
RLD	1	X’0C’	00 00 38	

Тип	ADOP	DLNOP	OPERAND	
TXT	00 00 38	4	00 00 00 08	

. . .

Тип	IDNUM	SYMBOL	SYMTYPE	OADR	DLINA	
ESD	1	EXAMP2	0	00 00 00	00 00 44	

Рис.11. Объектный модуль после трансляции исходного модуля EXAMP2

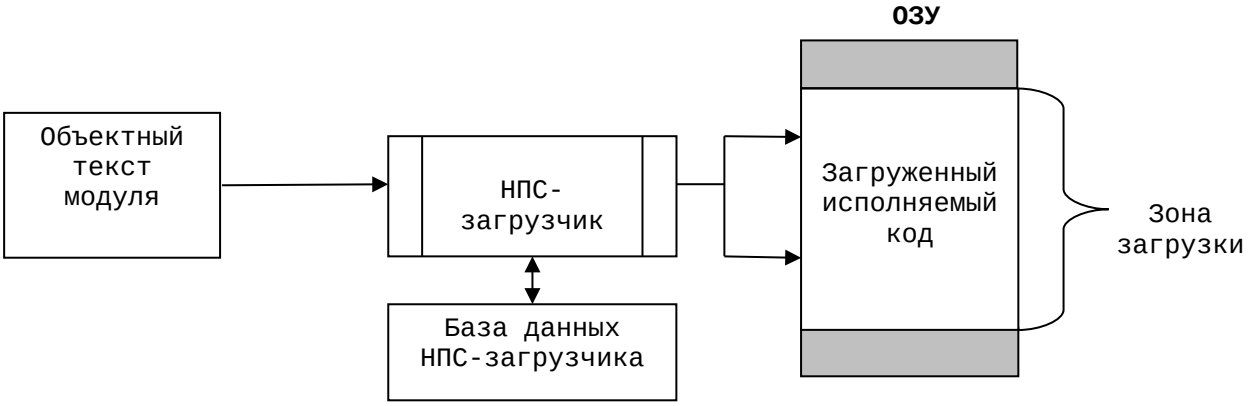

Рис.12. Общая схема функционирования НПС-загрузчика (односекционный случай)

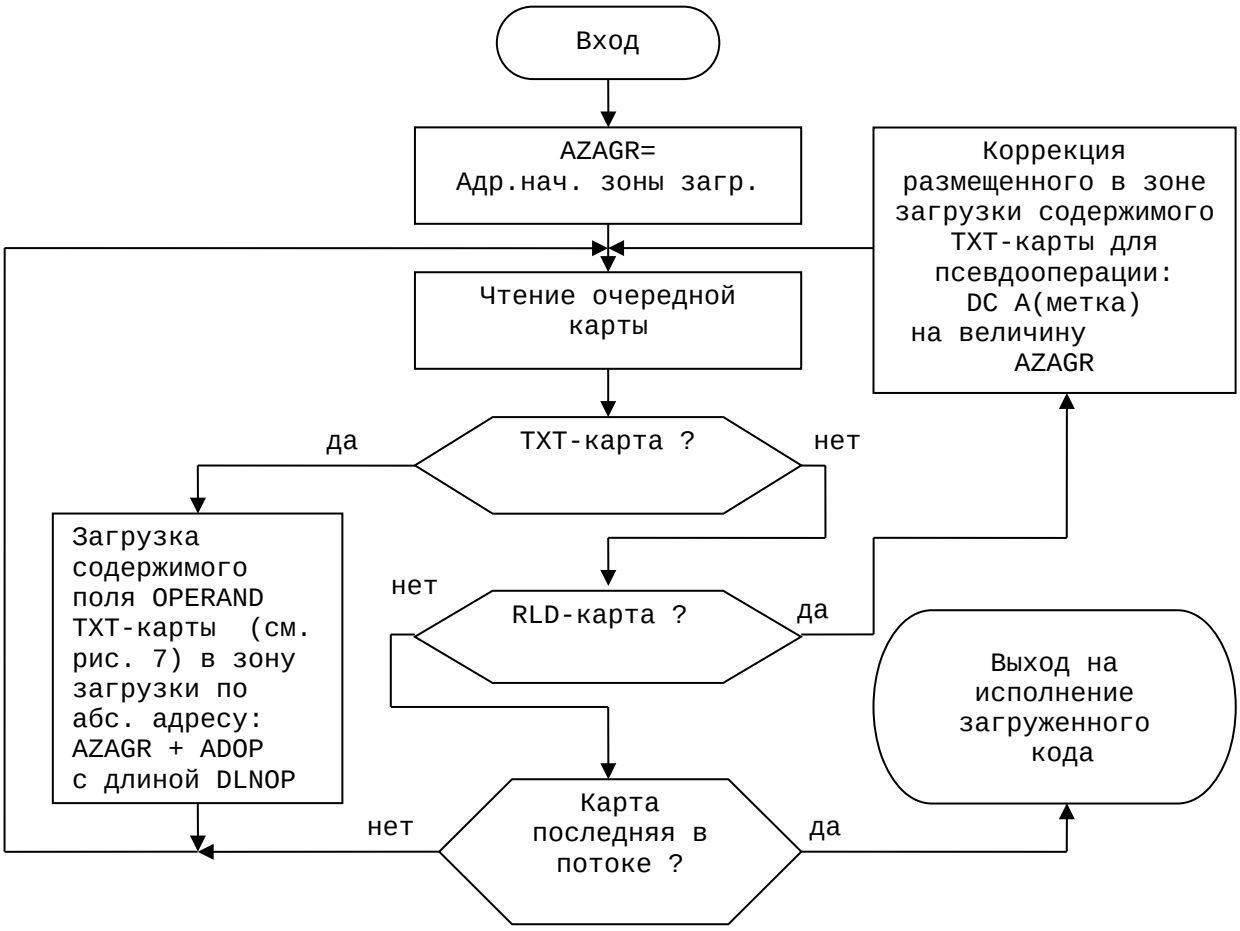

Рис.13. Алгоритм НПС-загрузчика для односекционного случая

2.1.4. Непосредственно – связывающий загрузчик (многосекционный случай)

Схема функционирования НПС-загрузчика для многосекционного случая приведена на рис.14.

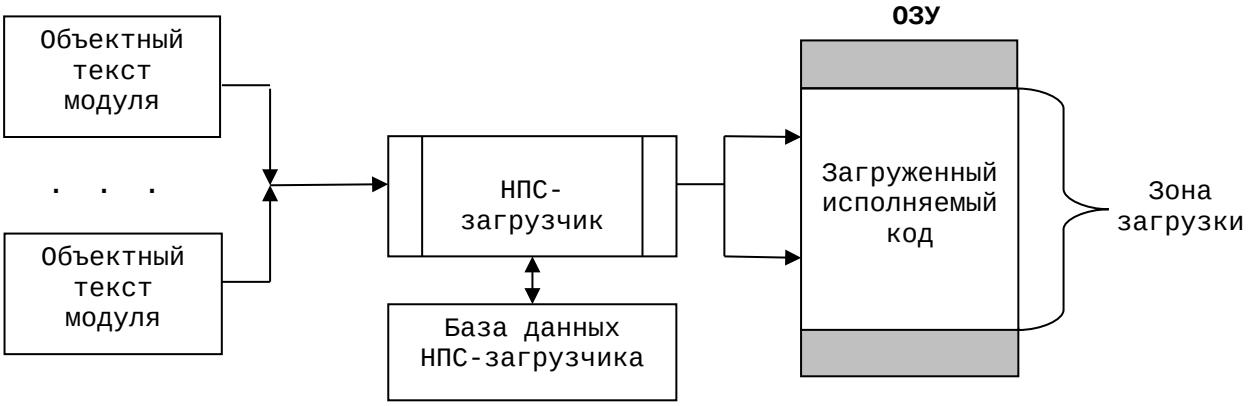

Рис.14. Общая схема функционирования НПС-загрузчика (многосекционный случай)

База данных НПС-загрузчика в этом случае включает в себя, как минимум, список объектных модулей и собственно объектные модули. Кроме того в БД входят как глобальная (**TABGVSYM), так** и локальная (**TABLVSYM),** таблицы внешних имен (символов).

Обоснуем необходимость введения названных таблиц.

Для этого, **во-первых,** вспомним, что внешнее имя (символ) используется для обеспечения модульного подхода к построению больших программных продуктов. При модульном подходе весь продукт получается путем компановки его из отдельных программных модулей относительно-небольшого размера. Очевидно, что при этом, например, в некотором модуле EXAMP1 (см. рис. 15) может возникнуть необходимость в использовании объектов с именем EXAMP2 и EXAMP3, объявленных в другом модуле EXAMP2:

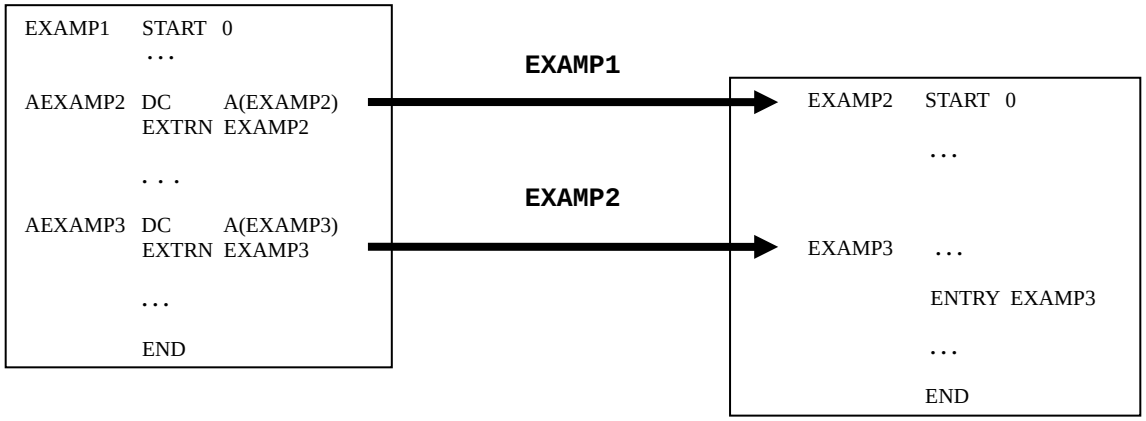

Рис.15. Пример межмодульных связей в многосекционном случае

Из приведенного примера видно, что доступ из программы EXAMP1 к внешним объектам с внешними именами/символами EXAMP2 и EXAMP3, объявленными в другой программе EXAMP2, можно построить на основе аппарата поименованной пары определение-ссылка. На рис. 15 эта пара изображена надписанной стрелкой, наконечник которой соответствует

определению, а оперение – ссылке. Обратим внимание на то, что в языке АССЕМБЛЕРА для **ссылки** на внешний объект (далее для краткости будем говорить внешнего имени или внешнего символа) используется псевдооперация **EXTRN**, а для **определения** внешнего объекта - псевдооперации **START** или **ENTRY**.

Во-вторых, отметим, что входной поток НПС-загрузчика образуется путем случайной конкатенации в единый пакет колод карт всех объектных модулей, подлежащих сборке.

Из сказанного следует, что при “неудачном” формировании входного потока (когда модуль EXAMP1, например, опережает в потоке модуль EXAMP2) разрешение внешних ссылок оказывается невозможным из-за того, что в момент обработки ссылок на внешние имена НПС-загрузчик не имеет информации об определениях этих внешних имен.

Для разрешения этой проблемы мы вынуждены будем заставить НПС-загрузчик делать вспомогательный **первый просмотр** входного потока с целью:

- выявления определений всех внешних имен путем селекции и анализа во входном потоке всех ESD-карт, порожденных псевдооперациями START и ENTRY,
- разметки зоны загрузки путем назначения каждому объявленному внешнему символу уникального абсолютного адреса в зоне загрузки,
- Записи каждой пары объявленный_внешний_символ-назначенный_ему_абсолютный_адрес в строку TABGVSYM.

В результате первого просмотра для нашего примера будет получена заполненная TABGVSYM следующего вида:

Имя внешнего символа	**Абсолютный адрес**
EXAMP1	A1
EXAMP2	A2
EXAMP3	A3

Рис. 16 Пример заполнения таблицы TABGVSYM.

На **втором просмотре** НПС-загрузчик должен будет решить следующие задачи:

- Загрузить в зону загрузки код каждого модуля из TXT-карт входного потока, согласно разметке, произведенной на первом просмотре,
- Настроить адресные константы, содержащие ссылки как на локальные, так и на внешние имена, используя для этого содержимое TABGVSYM.

Возможность решения первой из перечисленных задач второго просмотра очевидна. Для достижения успеха в решении второй задачи необходимо выполнение следующих требований к структуре входного потока:

- Входной поток должен соответствовать рис. 17,
- Структура каждого модуля в потоке должна соответствовать рис. 18.

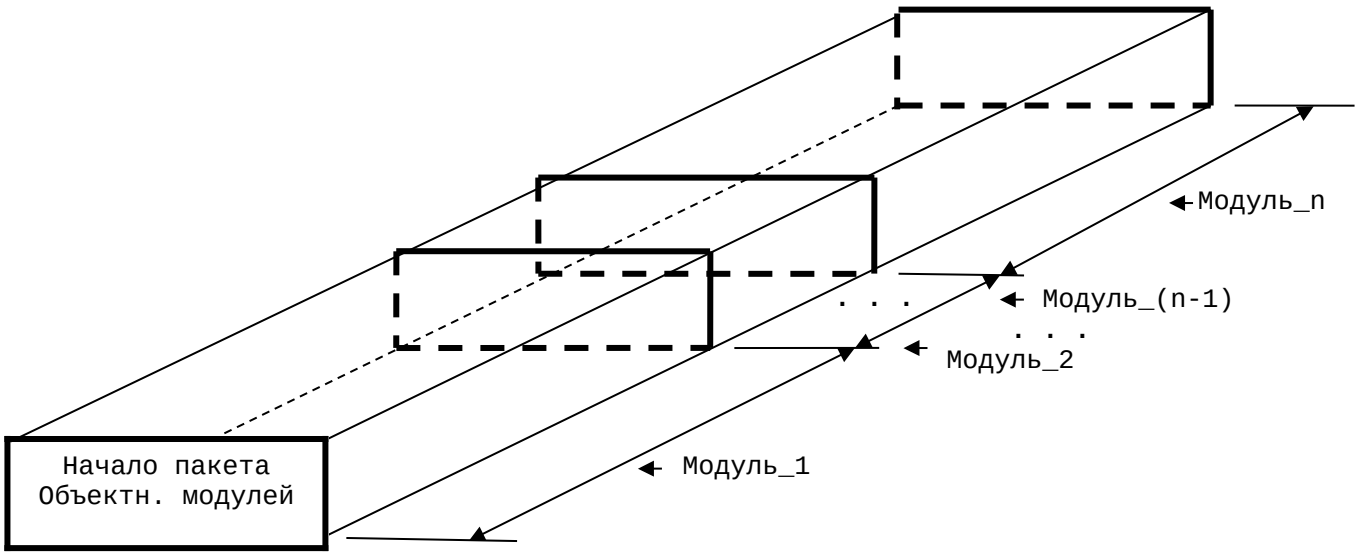

Рис.17 Структура входного пакета НПС-загрузчика

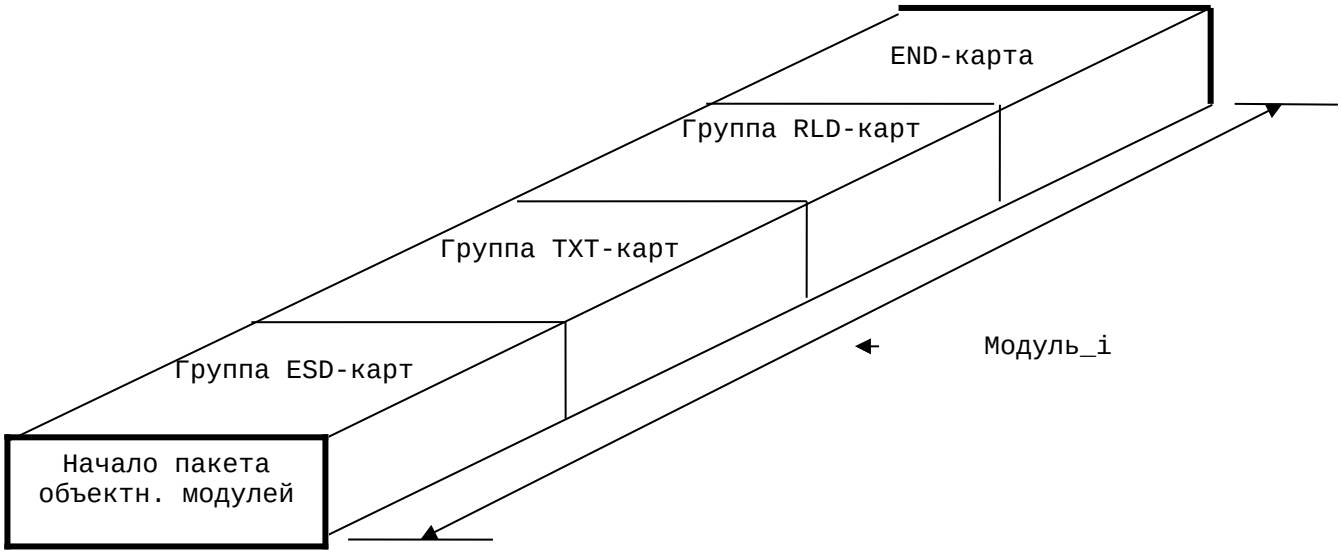

Рис.18 Структура отдельного модуля входного пакета НПС-загрузчика

Требование соответствия структуры объектного модуля образцу на рис. 18 продиктовано методикой расчета абсолютных адресов, являющихся значениями адресных констант, объявленных в программе.

Напомним, что, согласно этой методике, абсолютный адрес вычисляется по частям в два этапа:

- Сначала транслятором (результат остается на одной из TXT-карт),
- Затем НПС-загрузчиком при обработке соответствующей RLD-карты (результат остается в зоне загрузки).

RLD-карта (см. рис. 7 – рис. 9) может быть обработана при том условии, что TXT-карта и ESD-карта, на которые ссылается своими операндами (поля IDNUM и ADRSMECH) RLD-карта, уже просмотрены и обработаны НПС-загрузчиком до момента обработки данной RLD-карты. Это условие будет заведомо выполнено, если структура объектного модуля будет соответствовать рис. 18.

Обработка TXT-карты и ESD-карты, на которые ссылается RLD-карта, состоит в следующем:

- Хронологически первыми (см. рис. 18) обрабатывается ESD-карты. Суть обработки в том, что из ESD-карты (см. рис. 9) выделяется значение поля SYMBOL, которое используется далее как ключевое значение для поиска нужной строки в TABGVSYM. В найденной строке прочитывается значение абсолютного адреса спланированного для этого внешнего имени, которое далее переписывается в строку вспомогательной таблицы НПС-загрузчика TABLVSYM с номером, равным значению поля IDNUM ESD-карты. После обработки всей группы ESD-карт вся информация обо всех внешних именах, объявленных в данной программе, и абсолютных адресах этих внешних имен окажется в TABLVSYM.
- Обработка TXT-карты – носителя кода адресной константы, содержащей относительный адрес, соответствующий рассчитываемому абсолютному адресу, состоит в простой переписи ее операнда (поле OPERND см. рис. 7) в зону загрузки по нужному адресу (поле ADOP см. рис. 7) с учетом длины операнда (поле DLNOP см. рис. 7).

Из сказанного следует то, что в момент обработки RLD-карты, содержащей ссылки как на ESD-карту, так и на TXT-карту, прямого обращения к этим картам делать не надо, т.к. данные с этих карт окажутся соответственно в TABLVSYM (из ESD-карты) и в зоне загрузки (из TXT-карты). Это заставляет признать полезность таблицы TABLVSYM и включить ее очередным элементом в базу данных НПС-загрузчика.

Попутно заметим то, что дисциплина использования TABLVSYM предполагает ее обнуление перед обработкой очередного модуля входного потока и завершение заполнения после обработки группы ESD-карт текущего модуля входного потока.

При обработке входного потока из двух модулей EXAMP1 и EXAMP2 для рассмотренного выше примера TABGVSYM будет иметь вид, изображенный на рис. 16, а TABLVSYM после заполнения в процессе просмотра модуля EXAMP1 примет вид, как на рис. 19, а после заполнения в процессе просмотра модуля EXAMP2 – как на рис. 20.

	Абсолютный адрес
1	A1
2	A2
3	A3

Рис.19. Пример заполнения TABLVSYM. Обработка модуля EXAMP1

	Абсолютный адрес
1	A2
2	A3
3	

Рис.19. Пример заполнения TABLVSYM. Обработка модуля EXAMP2

Из рис. 10 и рис. 11 легко видеть то, что структуры объектных модулей программ EXAMP1 и EXAMP2 не удовлетворяют рис. 18, т.к. ESD-карты, TXT-карты и RLD-карты не отсортированы компилятором в однотиповые группы. Поэтому, для их сортировки мы будем вынуждены к уже спроектированным нами первому и второму просмотрам НПС-загрузчика добавить дополнительный нулевой просмотр, целью которого будет преобразование каждого модуля входного потока к каноническому виду, соответствующему рис. 18.

Алгоритм НПС-загрузчика

После добавления нулевого просмотра алгоритм НПС-загрузчика может быть представлен блок-схемами на рис. 21 – 24.

Подводя итог рассмотрения НПС-загрузчика, перечислим его достоинства и недостатки.

Достоинства:

- Сравнительно малый размер загрузчика оставляет больше место в зоне загрузки для исполняемого кода прикладных программ.
- Отсутствие необходимости повторной компиляции отлаженных программ.
- Возможность компановки программ, написанных на различных языках программирования.
- Высокая степень автоматизации всех задач из списка общих задач загрузчика.

Недостатки:

Полная автоматизация решения всех общих задач загрузки делает невозможным выполнение необходимого в некоторых случаях ручного управления планированием размещения модулей в зоне загрузки.

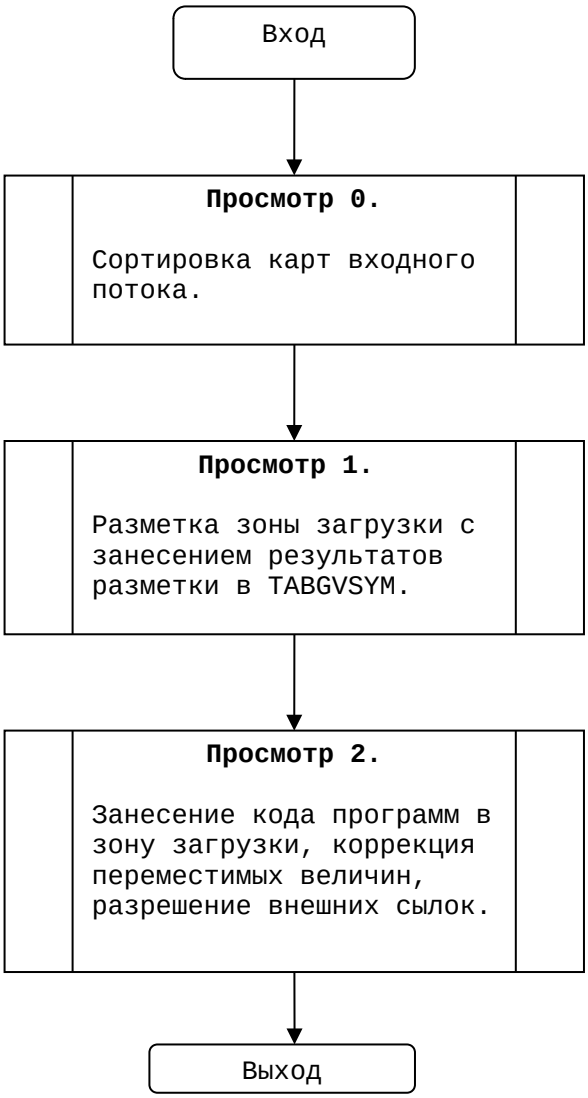

Рис.21. Макро-блок-схема НПС-загрузчика

В ы в о д ы :

- НПС-загрузчик автоматизирует решение всех четырех общих задач загрузки,
- следовательно, по степени автоматизации реализует наиболее совершенную схему загрузки и числа рассмотренных схем.

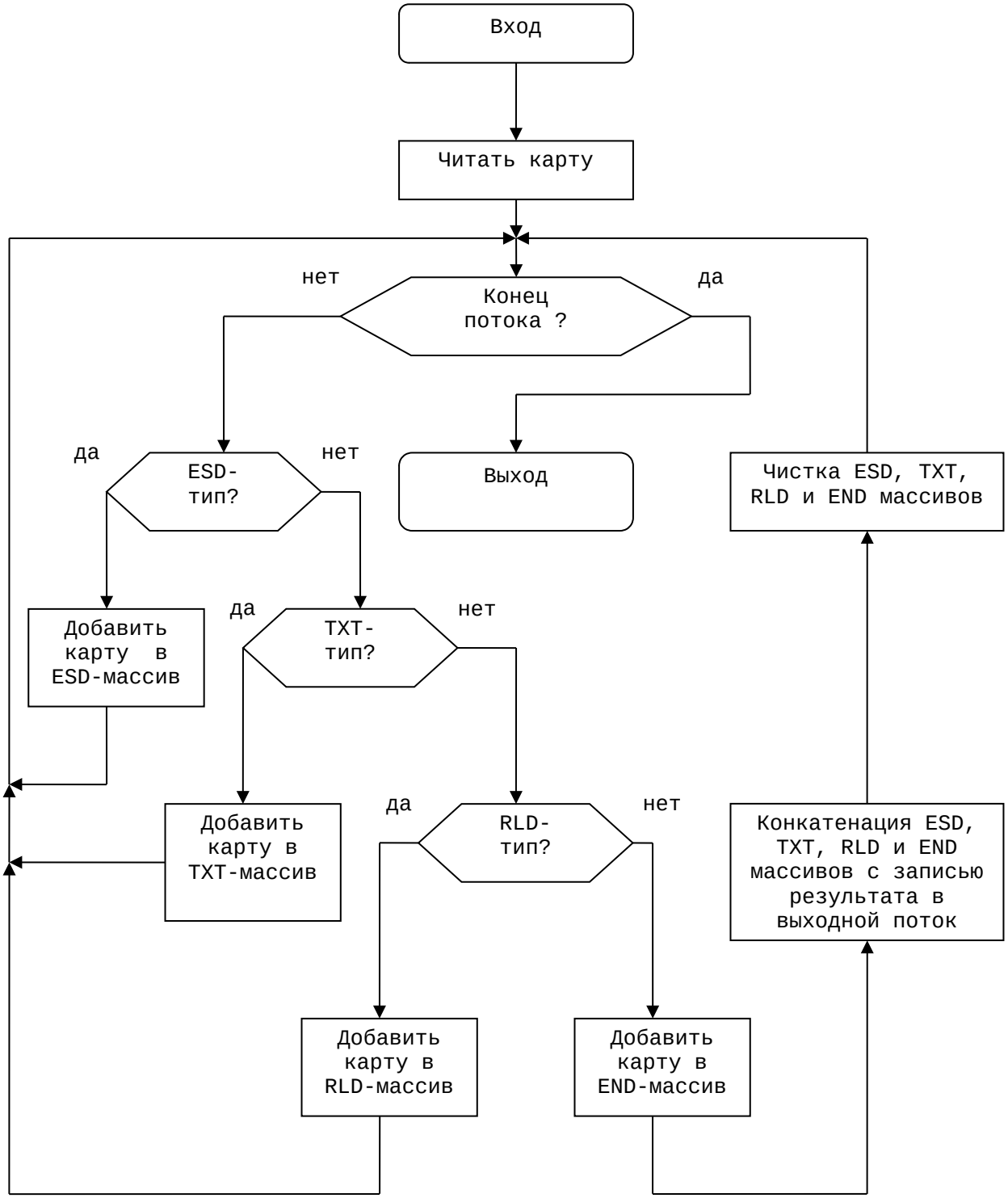

Рис.22. Алгоритм нулевого просмотра НПС-загрузчика

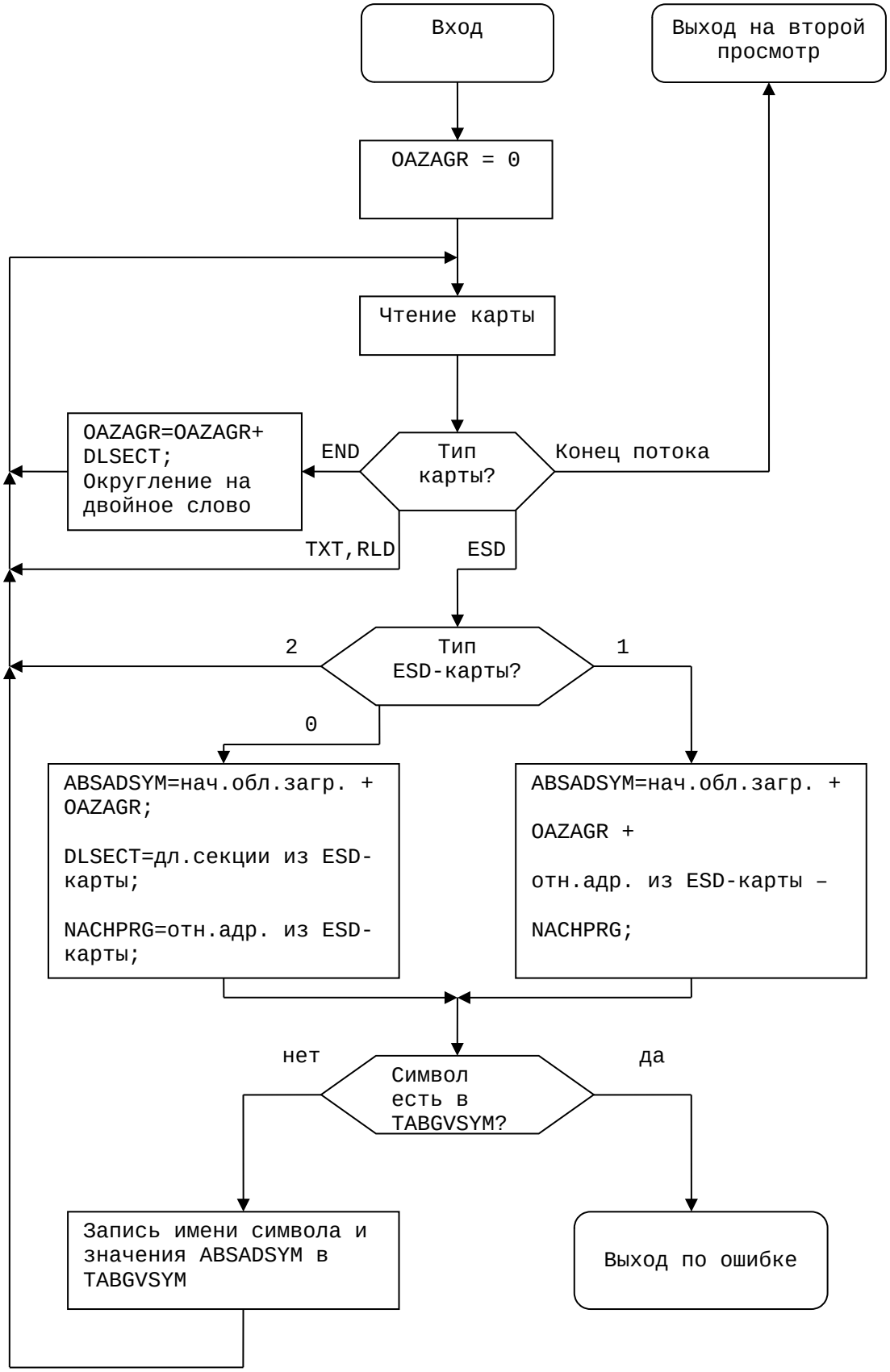

Рис.23. Алгоритм первого просмотра НПС-загрузчика

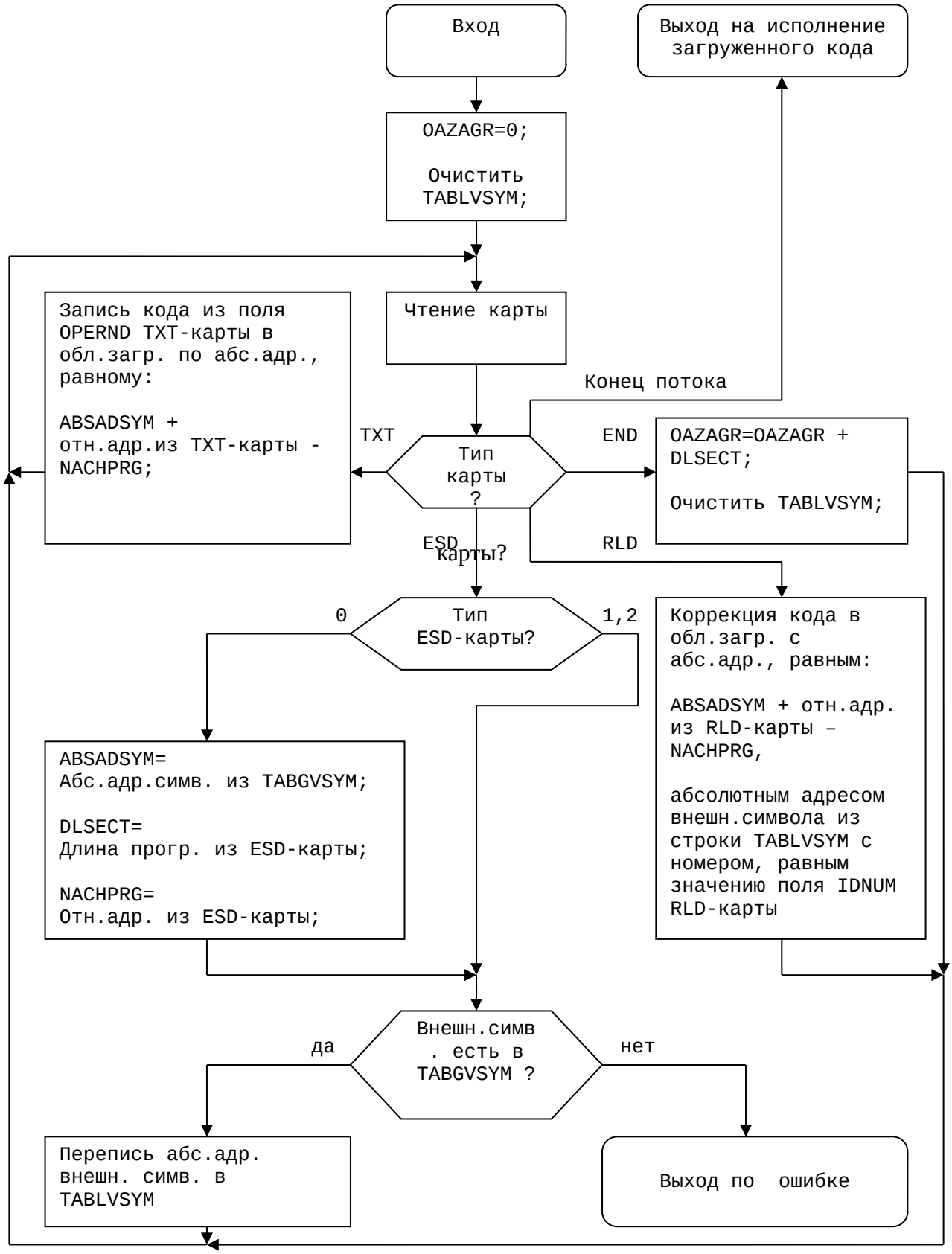

Рис.24. Алгоритм второго просмотра НПС-загрузчика

2.2. Классификация схем загрузки

Мы рассмотрели загрузчики со следующими схемами загрузки:

- Загрузка способом компиляция-выполнение.
- Абсолютная загрузка.
- НПС-загрузка.

Загрузчик любого из перечисленных типов с различной степенью автоматизации решает полный набор общих задач загрузки:

1. Распределение памяти зоны загрузки под загружаемые модули.
2. Разрешение ссылок на внешние символы в компануемых программах.
3. Настройка переместимых (т.е. зависящих от места загрузки) величин.
4. Инициализация выполнения кода, сформированного в зоне загрузки.

Общим свойством всех этих схем было то, что при их использовании мы предполагали **полное решение всех четырех общих задач загрузки до момента инициализации загруженного кода**. Схемы с таким свойством называются **<u>статическими</u>** в противоположность **<u>динамическим</u>**, для которых характерна **инициализации загруженного кода без полного решения всех четырех общих задач загрузки.**

При использовании статического загрузчика во время его работы ОЗУ ЭВМ распределено так, как показано на рис.25.

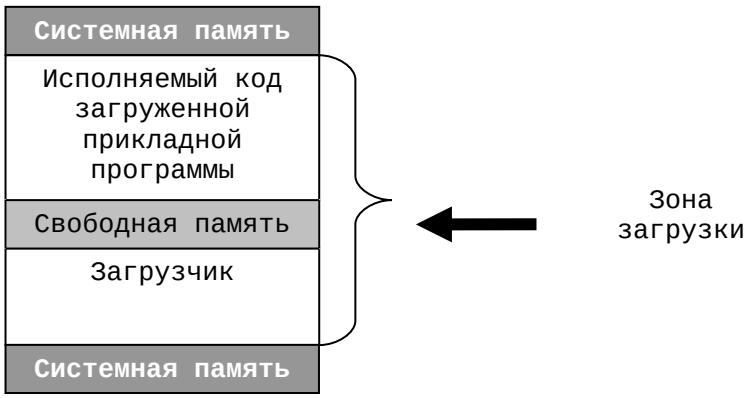

Рис.25. Распределение ОЗУ при работе загрузчика

Из рис. 25 видно, что часть ОЗУ д.б. отведена под код загрузчика и, следовательно, при этом сокращается часть зоны загрузки для прикладной программы. Это сокращение объема зоны загрузки может оказаться слишком значительным, чтобы им пренебрегать. Сократить размер кода загрузчика и, соответственно, увеличить свободную часть зоны загрузки можно путем разделением загрузчика на две части:

- Первую, которая, решая первые три из общих задач загрузки, результирующий код заносит не в зону загрузки, а в файл,

- Вторую, которая решает 4 общую задачу загрузки, читая код из файла.

П р и м е ч а н и е:

Отметим, что указанный выше способ разделения загрузчика на две части не является единственным. Так, например, в сложных динамических схемах загрузки в первую часть загрузчика, работающую до инициализации исполняемого кода проекта, включают единственную задачу с номером 1 из полного набора общих задач загрузки, т.е. задачу распределение памяти зоны загрузки под загружаемые модули, написанные пользователем. Все оставшиеся задачи с номерами от 2 и выше выполняются после инициализации и на фоне работы исполняемого кода проекта.

В рассмотренном выше способе разделения загрузчика в момент работы второй части загрузчика распределение ОЗУ будет похожим на рис. 25, но потери размера зоны загрузки значительно сократятся. Образовавшиеся при делении две части загрузчика носят самостоятельные названия. Первая называется **с т а т и ч е с к и м о б ъ е д и н и т е л е м**, а вторая – **и н и ц и а т о р о м**. В том случае, если статический объединитель формирует абсолютный загружаемый код (т.е. код, жестко привязанный к абсолютным адресам зоны загрузки), его называют **п о с т р о и т е л е м образа памяти**, а в противном случае – **р е д а к т о р о м с в я з е й.**

Собрав все сказанное вместе, получим классификационную схему для схем загрузки (см. рис. 26).

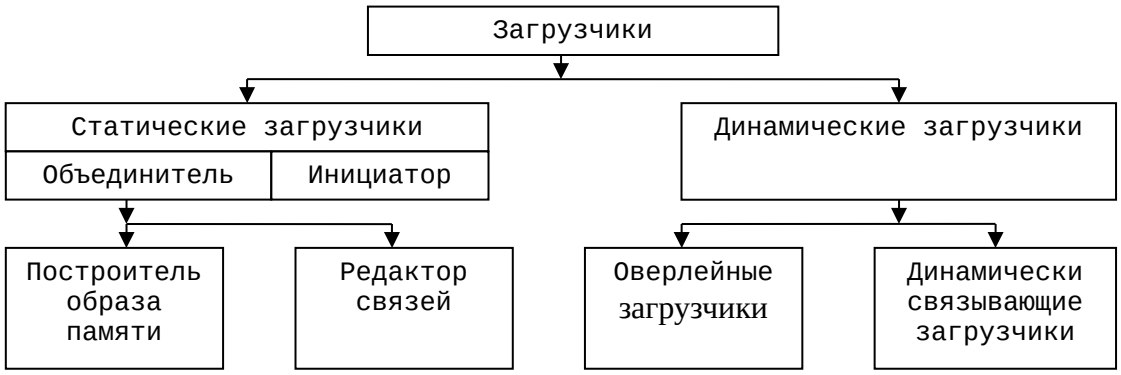

Рис.26. Разновидности схем загрузки

2.3. Примеры статических загрузчиков

Приведем несколько примеров построения различных системных обработчиков, изображенных на рис. 26. Строить эти обработчики будем из системных утилит операционной системы ПДО (Подсистема Диалоговой Отладки), широко используемой на ЭВМ IBM 360/370.

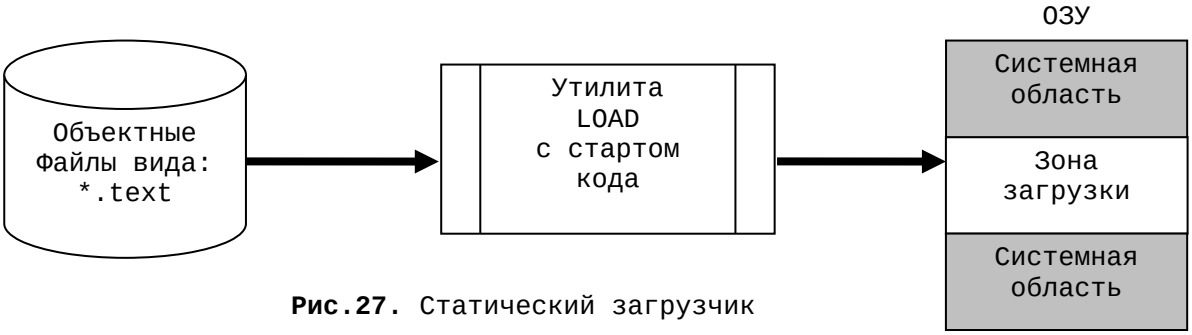

Рис.27. Статический загрузчик

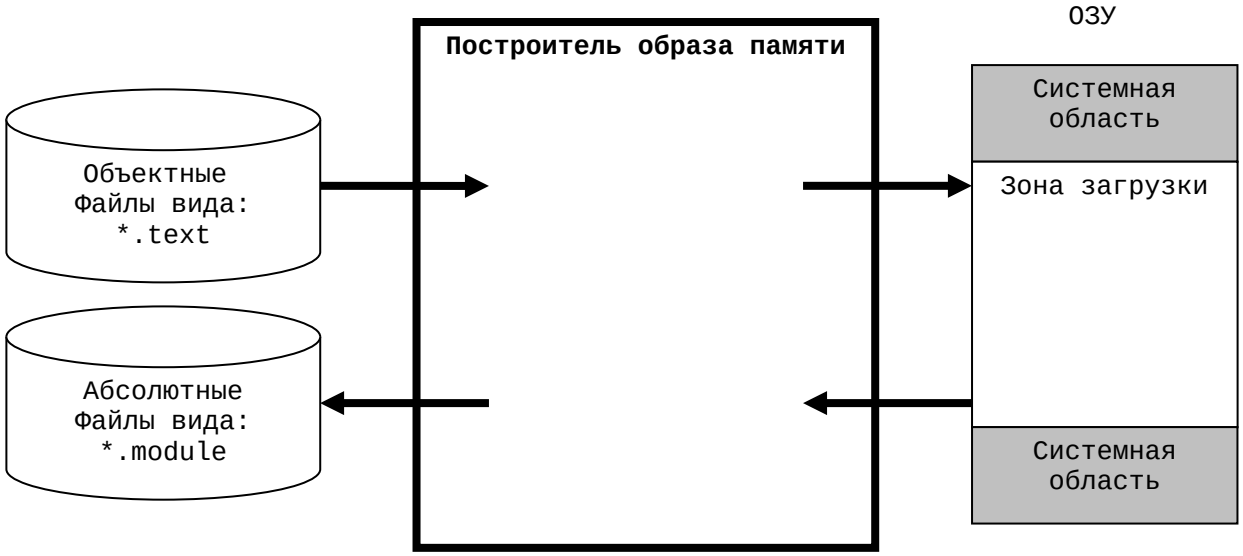

Рис.28. Построитель образа памяти

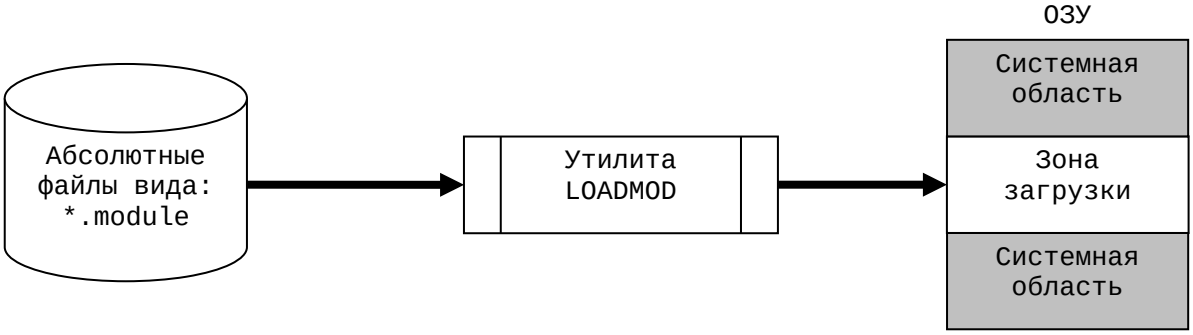

Рис.29. Инициатор исполняемого кода

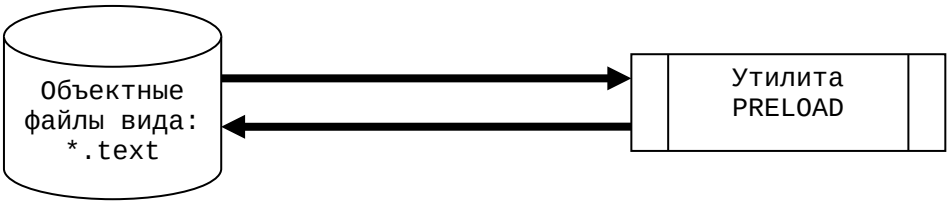

Рис.30. Редактор связей

Рис.31. Редактор связей

2.4. Динамические загрузчики

2.4.1. Оверлейный загрузчик

История использования ЭВМ сравнительно коротка и в начале этого исторического пути использовались ЭВМ, имеющие небольшую оперативную память и невысокое быстродействие.

Так, например, всего пару десятков лет тому назад достаточно широко использовались ЭВМ IBM PC, работающие в реальном режиме (real mode) с применением однозадачной и однопользовательской операционной системы MSDOS, которая предполагала, что исполняемый код после его формирования отправлялся на хранение в файловую систему на внешний диск.

Для организации сеанса выполнения исполняемый код д.б. загружен в зону загрузки в ОЗУ, имеющей очень скромный размер. Так в рассматриваемой конфигурации в ЭВМ IBM PC объем ОЗУ составлял 1 мб., из которого около 400 кб. занимала сама MSDOS. На приложения оставалось всего 600 кб. свободного ОЗУ, что делало возможным загрузку и выполнение только небольших приложений с размером, меньшим 600 кб.

Для проектов с общим объемом более, чем 600 кб., предлагалось подготавливать исполняемый код проекта в **оверлейном формате**, который предусматривал деление всего проекта на куски (сегменты), подлежащие хранению на диске. Один из сегментов, начинающий решение задачи имел статус корневого (root) сегмента, и д.б. находится в зоне загрузки в ОЗУ на протяжении решения всей задачи. Остальные сегменты подгружались в ОЗУ и вытеснялись из него по мере необходимости в ходе решения задачи.

Принцип деления на сегменты был тесно связан с принципом рекурсивного использования понятий **вызывающая** программа и **вызываемая** подпрограмма. Именно эти программы и подпрограммы, проектируемые (не всегда безупречно) самим программистом, выступали в качестве сегментов.

Для осознания идеи, особенности и границ эффективности динамической схемы загрузки проектов оверлейного типа рассмотрим следующий пример.

Пусть нам требуется собрать между собой для выполнения пять программ (объем в килобайтах каждой из программ указан в скобках за идентификатором):

A(20k), B(20k), C(30k), D(10k), E(20k).

Любая статическая схема даст в результате исполняемый код с объемом в 100к (см. рис. 32).

Допустим, что межпрограммные вызовы, имеющие синхронный тип, т.е. являющиеся по своей сути передачами управления из вызывающей программы в вызываемую подпрограмму с ожиданием возврата, соответствует стрелкам на рис. 32. Тогда с учетом рекурсивной вложенности межпрограммных вызовов мы получим следующее изображение структуры межпрограммных вызовов (иначе - графа управления) в исполняемом коде (см. рис. 33).

Из рис. 33 видно, что:

- Программы по управлению раскладываются на 3 уровня,
- Учитывая невозможность одновременного вызова двух программ на одинаковом уровне из-за синхронного свойства вызовов, мы можем планировать в зоне загрузки для всех программ одинакового уровня свою отдельную **общую память с размером, равным размеру самой длинной программы уровня**,
- Максимальная память потребуется только при самом глубоком по вложенности вызове и будет равной 70к, что на треть меньше, чем 100к при статической сборке.

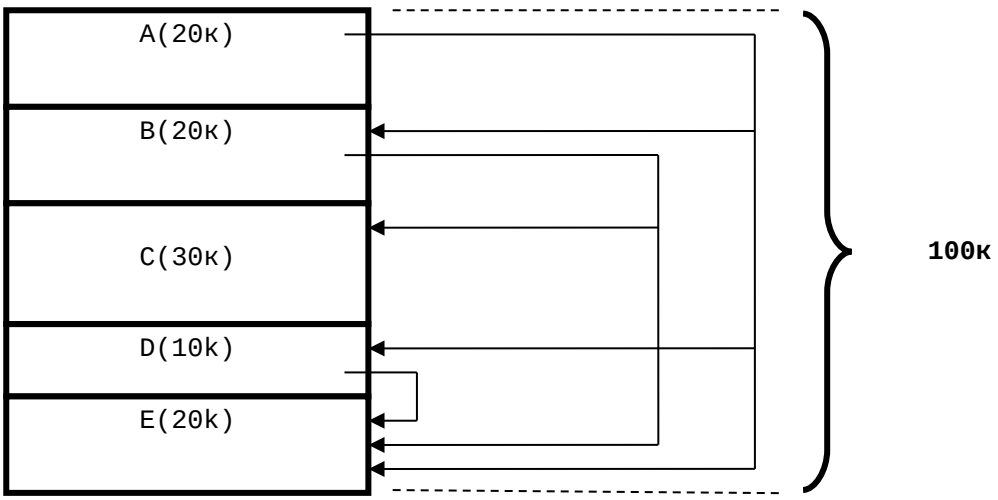

Рис.32. Результат статической сборки

Если обеспечить динамическую подгрузку нужной программы каждого уровня при исполнении оверлейного исполняемого кода в общую память данного уровня, то можно получить заметную экономию суммарной памяти, но, разумеется, с некоторым одновременным замедлением скорости выполнения межпрограммных вызовов из-за неизбежных подгрузок в ОЗУ нужных и вытеснений из ОЗУ неактуальных сегментов оверлейного кода.

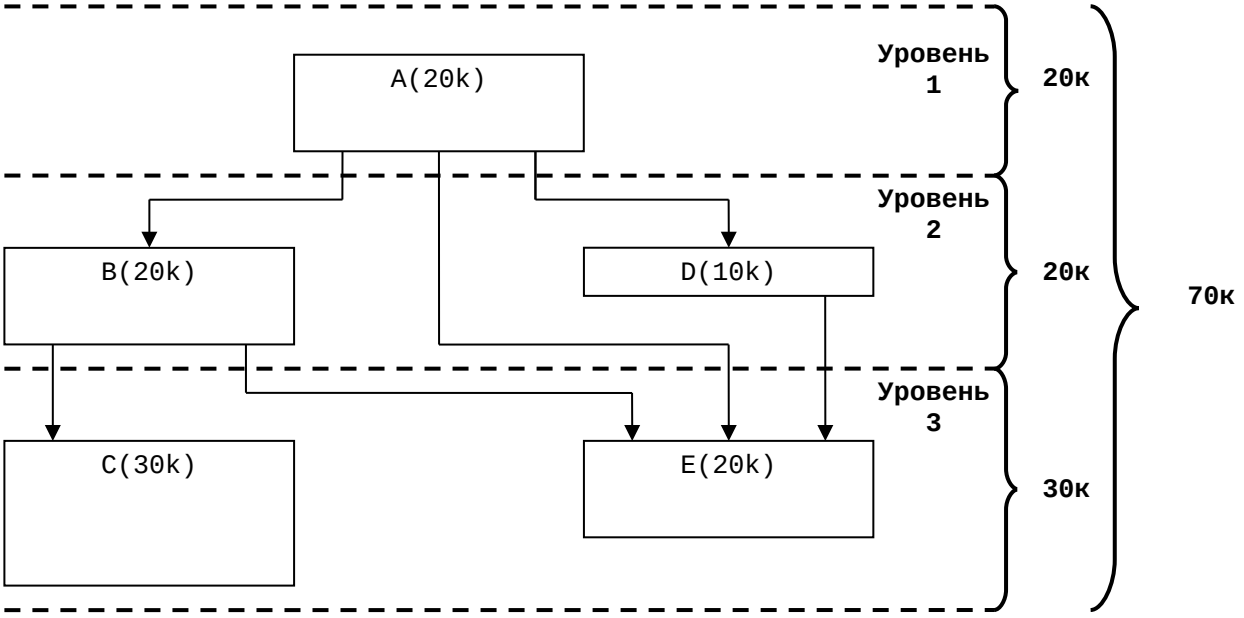

Рис.33. Трехуровневая оверлейная структура

Перечислим общие соображения, которые следует использовать при реализации механизма динамического вызова сегментов оверлейного кода:

1. На всех этапах планирования и исполнения оверлейного кода следует исходить из того, что для его выполнения требуется участок зоны загрузки равный сумме размеров всех уровней оверлейного кода (в рассмотренном примере – 70к), независимо от реальной эффективности использования памяти этого участка.
2. Загрузка (вытеснение) сегмента оверлейного кода в память соответствующего уровня в зоне загрузки оверлея является сервисом операционной системы (т.е. системным действием), который предоставляется специальным менеджером операционной системы по специальному запросу на это системное действие.
3. Запросы на указанное выше системное действие должны поступать из программ оверлейного кода согласно графу управления, в котором прямые обращения из программы в подпрограмму замещаются на эти запросы.
4. Замещение межпрограммных обращений на запросы выполняется либо компилятором, либо специальной системной утилитой - динамическим оверлейным загрузчиком до момента старта оверлейного кода.
5. Таким образом, список средств поддержки механизма динамического вызова сегментов оверлейного кода состоит из двух элементов:
   - оффлайнового динамического оверлейного загрузчика,
   - онлайнового менеджера подгрузки сегментов.

Подводя итог рассмотрения оверлейного загрузчика, перечислим его достоинства и недостатки.

Д о с т о и н с т в а:

- Обеспечение **принципиальной** возможности выполнения проектов любых размеров на ЭВМ с небольшим ОЗУ, однако, оснащенных дисковой памятью достаточного размера.
- Исключение необходимости повторной компиляции отлаженных программ.
- Возможность компановки программ, написанных на различных языках программирования.

Н е д о с т а т к и:

- В случае возможности построения исполняемого кода проекта как в статическом, так и в оверлейном варианте очевиден проигрыш по скорости выполнения оверлейного варианта в сравнении со статическим.
- Если проект имеет несколько вариантов функционирования, зависящих от стартовых данных, то не факт, что в конкретном сеансе работы по одному из конкретных вариантов граф управления будет развернут на максимальную глубину вложенности, что ведет к неэффективному использовании памяти зоны загрузки, зарезервированной всегда на максимальную глубину вызовов.
- Необходимость ручного построения многоуровневого графа управления, что при больших и сложных графах может стать неподъемной задачей для программиста.

В ы в о д ы:

Оверлейный загрузчик:

- обеспечивает принципиальную возможность выполнения проектов, размер которых превышает размер зоны загрузки ЭВМ,
- может быть использован в проектах с графами управления разумных размеров, допускающих их ручное построение,
- оверлейный код, подготовленный с помощью оверлейного загрузчика, работает медленнее, чем статический код,
- оверлейный код по-прежнему актуален. Применение оверлейных решений при построении обычных программных проектов сегодня сравнительно редкое явление, т.к. за последние годы произошел заметный прогресс развития вычислительных средств, который привел к резкому (на порядки) увеличению размера памяти (как оперативной, так и дисковой) и быстродействия. Вместе с тем, одна из ветвей того же самого прогресса, связанная с разработкой многоядерных процессоров, в состав которых включены, например, как ядро обычного, так и ядра сигнальных (DSP) процессоров, которые информационно связаны через общую память малого размера вновь сделало актуальной проблему построения оверлейного кода.

2.4.2. Динамически связывающий загрузчик

Область применения динамически связывающих загрузчиков начинается там, где:

- проявляется нехватка оперативной памяти для требуемого объема вычислений. Сегодня это может иметь место не только на маломощных специализированных микропроцессорах, но даже на мощных ЭВМ в случаях использования на них концепций многопользовательских (многозадачных) моделей вычислительного процесса,
- эффект нехватки оперативной памяти не может быть скомпенсирован применением оверлейных решений из-за недостатков оверлейного загрузчика, связанных с возможной неэффективностью использования зарезервированного участка зоны загрузки или большой сложностью графа управления, которая не допускает его ручное исследование и, соответственно, ручное построения системы оверлейных уровней.

Основным недостатком всех ранее рассмотренных схем загрузки является то обстоятельство, что, например, если в исходном тексте программы программист закодировал вызов (обращение) к некоторой подпрограмме, то даже в том случае, когда эта подпрограмма вообще не используется в сеансе работы программы (например, в сеансе вызов обходится при анализе некоторого внутреннего условия), загрузчики, рассмотренные выше, все равно выполняют лишнюю для данного сеанса работу по планированию памяти под неиспользуемую подпрограмму и разрешению ссылок из программы на эту подпрограмму.

Более того, все рассмотренные схемы требовали от программиста явного перечисления подпрограмм, которые могут потребоваться его программе. При такой постановке вопроса оказывается невозможным сделать программы, которые не располагают полным списком вызываемых подпрограмм, в момент написания программы, а уточняют этот список на этапе выполнения программы по данным приходящим, например, извне.

Механизм динамического связывания устраняет основной недостаток, указанный выше. Сам механизм имеет две части: **клиентскую** и **серверную**.

Суть **клиентской** части в том, что:

- из четырех задач списка общих задач загрузки сначала выполняется только первая, да и та, только для основной (main) программы проекта. Другими словами происходит планирование памяти в зоне загрузки под тело основной программы и ее последующая загрузка.
- После происходит старт кода основной программы и выполнение участка траектории этой программы до точки, где необходимо получение абсолютного адреса, либо подпрограммы для последующей передачи в нее управления, либо внешней переменной для последующих ее чтения и (или) записи.
- В момент достижения точки траектории выполнения, указанной выше, происходит временное блокирование основной программы

и выдается запрос (системный вызов) к операционной системе на сервис, состоящий в расчете требуемого абсолютного адреса.

- После завершения расчета абсолютный адрес возвращается из операционной системы клиенту – основной программе, для завершения передачи управления, чтения или записи (т.е. здесь происходит решение второй и третьей задач из списка общих задач загрузки).
- После передачи управления из вызывающей программы в подпрограмму эта подпрограмма наследует роль клиента для получения права делать запросы на расчет абсолютных адресов собственных подпрограмм и внешних переменных.

Суть **серверной** части в том, что соответствующий менеджер операционной системы:

- Получив запрос на расчет абсолютного адреса подпрограммы или внешней переменной, производит проверку присутствия в зоне загрузки сегмента кода, содержащего указанную в запросе подпрограмму или внешнюю переменную.
- Если сегмент кода с указанной в запросе подпрограммой или внешней переменной находится в зоне загрузки, то значение требуемого абсолютного адреса известно. Следовательно, оно м.б. возвращено клиенту.
- Если сегмента кода с указанной в запросе подпрограммой или внешней переменной нет в зоне загрузки, то при наличие свободного места в зоне загрузки этот сегмент загружается туда (т.е. решается первая задача из списка общих задач загрузки относительно сегмента), абсолютные адреса, требуемых подпрограммы или внешней переменной, рассчитываются (т.е. решается вторая и третья задачи из списка общих задач загрузки) и возвращаются клиенту.
- В случае, если при выполнении предыдущего действия обнаружено отсутствие свободного места в зоне загрузки, то происходит его формирование путем вытеснения неактуальных сегментов и далее повторятся предыдущее действие.

Подводя итог рассмотрения динамически связывающего загрузчика, перечислим его достоинства и недостатки.

Д о с т о и н с т в а:

- Обеспечение **принципиальной** возможности выполнения проектов любых размеров на ЭВМ с небольшим ОЗУ, однако, оснащенных дисковой памятью достаточного размера.
- Исключение необходимости повторной компиляции отлаженных программ.
- Возможность компановки программ, написанных на различных языках программирования.
- Эффективное использование памяти зоны загрузки.

Н е д о с т а т к и:

- В случае возможности построения проекта как в статическом, оверлейном, так и в динамически связываемом варианте очевиден проигрыш последнего по скорости выполнения как оверлейному варианту, так и статическому варианту.

В ы в о д ы:

динамически связываюший загрузчик:

- обеспечивает принципиальную возможность выполнения проектов, размер которых превышает размер зоны загрузки ЭВМ,
- может быть использован в проектах с графами управления любых размеров,
- код, подготовленный с помощью динамически связывающего загрузчика, работает медленнее, чем статический и оверлейные коды,
- динамически связываемый код чрезвычайно актуален и применяется как основой вариант на современных мощных ЭВМ в случаях использования на них концепций многопользовательских (многозадачных) моделей вычислительного процесса.

3. РЕКОМЕНДАЦИИ ПО ПОРЯДКУ ВЫПОЛНЕНИЯ РАБОТЫ

Данная лабораторная работа предполагает использование в качестве начальной основы отлаженной и работающей программы-макета, т.е. утилиты, содержащей:

- абсолютный загрузчик,
- эмулятор ЭВМ IBM 370,
- низкоуровневый отладчик программ, выполняющихся на эмуляторе.

По степени функциональности утилита способна демонстрировать работоспособность законченной правильной программы контрольного примера. Программа-макет написана на алгоритмическом языке Си и снабжена подробным контекстным комментарием.

Студенту предлагается, если необходимо, расширить первичный набор машинных операций эмулятора ЭВМ IBM 370, поддерживаемый утилитой, до уровня, позволяющего выполнить на эмуляторе пример, содержащийся в его личном задании, и произвести соответствующую доработку утилиты. При этом необходимо:

- ознакомиться с исходными текстами утилиты и контрольного примера, на который настроена утилита,
- проверить работоспособность утилиты путем проведения сборки из объектных модулей контрольного примера его исполняемого кода и запуска этого кода на выполнение с помощью эмулятора и отладчика утилиты,
- составить список машинных инструкций ЭВМ IBM 370, который необходим для выполнения примера, содержащегося в личном задании, и, одновременно, не поддержанный эмулятором,
- спроектировать перечень коррекций утилиты с целью реализации поддержки списка машинных инструкций ЭВМ IBM 370, составленный в предыдущем пункте,
- ввести коррекции в исходный текст утилиты, снабдив измененные места исходного текста построчным комментарием, и записать модифицированный вариант утилиты в свой личный директорий на сервере,
- выполнить отладку модифицированной утилиты,
- составить отчет по результатам работы.

4. ТРЕБОВАНИЯ К СОДЕРЖАНИЮ ОТЧЕТА

Отчет должен соответствовать рекомендациям по оформлению студенческих курсовых работ, размещенных на сайте университета, и в общем случае должен содержать следующие разделы:

- описания примера из личного задания (указать формулировку личного задания),
- постановки задачи модификации программы-макета (описание выбранного способа модификации программы-макета),
- содержащие перечень и пояснения модификаций состава и (или) структуры БД утилиты,
- перечень и пояснения модификаций алгоритма утилиты (возможные формы описаний в виде: блок-схемы, текста на Си с комментариями, а в очевидных случаях обычным текстом),
- выводы по результатам работы (указать позитивные и негативные моменты, проявившиеся в процессе работы, с общей личной оценкой проделанной работы).

5.ТРЕБОВАНИЯ К КОМПЛЕКТАЦИИ, ОФОРМЛЕНИЮ И ПРЕДЪЯВЛЕНИЮ РЕЗУЛЬТАТОВ КУРСОВОЙ РАБОТЫ

К результатам курсовой работы относятся:

- Отчеты в бумажном исполнении,
- Отчеты в электронном исполнении,
- Исходные тексты компилятора с ЯВУ,
- Исходные тексты компилятора с АССЕМБЛЕРА,
- Исходные тексты загрузчика.

Сводный отчет по курсовой работе “Системы программирования”, выполненной студентом, должен:

- быть укомплектован из отчетов по всем элементам разработанной учебной системы программирования,
- по своей структуре соответствовать рис.34,
- быть подготовлен как в бумажном, так и в электронном вариантах исполнения,
- быть передан преподавателю после завершения курсовой работы в момент защиты результатов курсовой работы.

Работы по отдельным элементам разрабатываемой учебной системы программирования проводятся в течение семестра. Отчеты по каждому элементу:

- составляются по завершению работы над его реализацией,
- предъявляются преподавателю в течении семестра,
- включаются в сводный отчет к концу семестра.

Исходные тексты элементов учебной системы программирования вместе с электронным вариантом сводного отчета оформляются и передаются преподавателю в виде архивного файла, рекомендуемый состав папок и файлов которого приведен на рис.35. В названии архива **Var_XX**, вместо **XX** следует использовать номер варианта своего задания на курсовую работу.

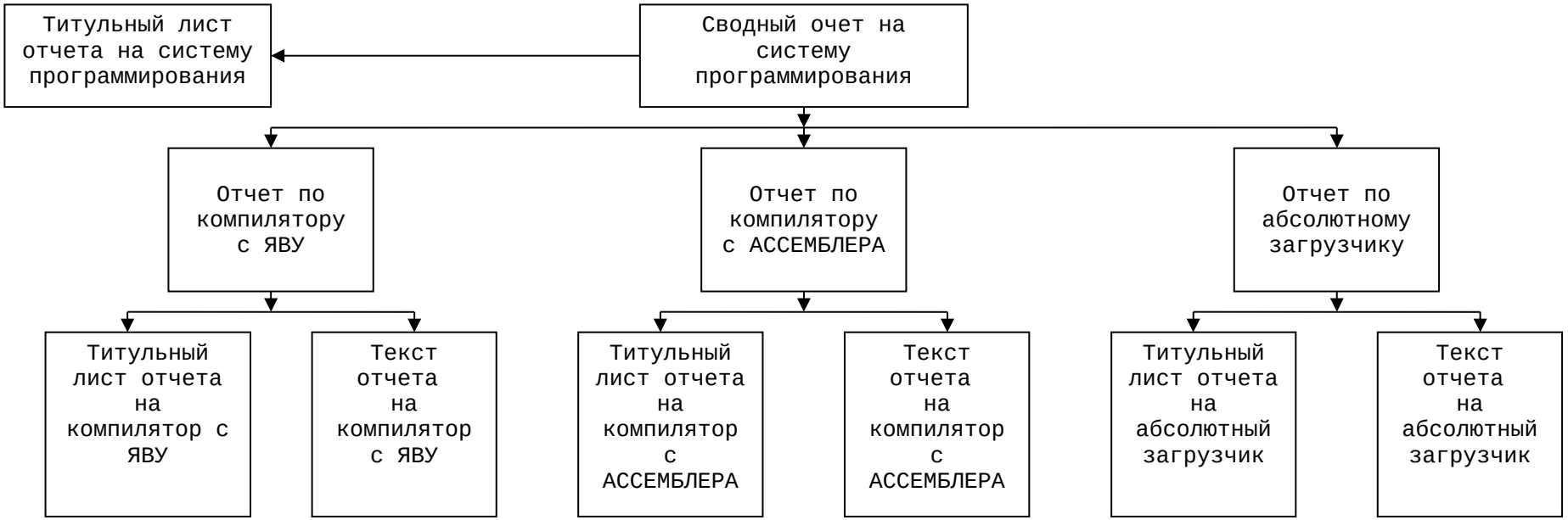

Рис.34. Структура сводного отчета по курсовой работе

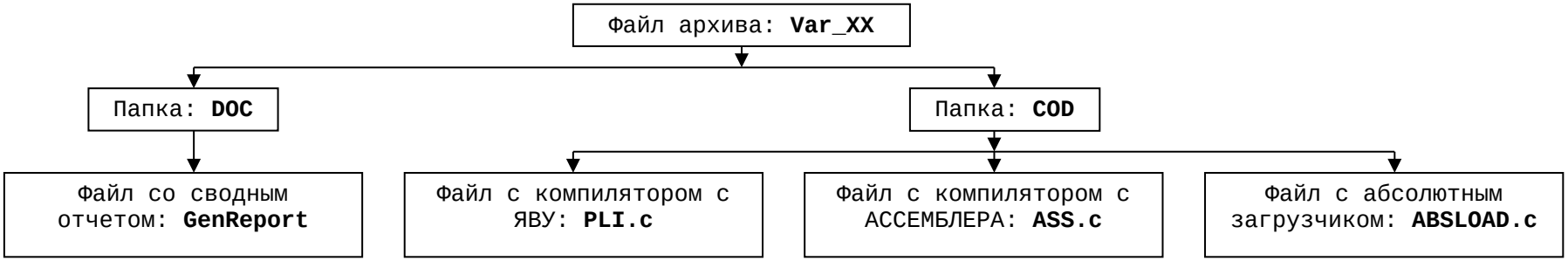

Рис.35. Рекомендуемый состав архива